

Recent Advances in the Application of AI and Machine Learning to Medical Video Analysis Systems

Marios S. Pattichis, FEAMBES, Senior Member, National Academy of Inventors

Director of online M.Sc. In Computer Engineering in Applied Machine Learning And AI Systems Engineering

Department of Electrical and Computer Engineering

The University of New Mexico

Talk Outline

- Introduction
- Background: Common Architectures for Echocardiography
- Foundational Material:
 - Geometric transformations from imaging
 - Digital video basics
 - Working with limited datasets: basic techniques
 - An Introduction to Learning
 - Transformer Architectures
- A Foundation Model for Endoscopy
- Generative models based on diffusion
- Generative Adversarial Network (GAN) and Domain Adaptation
- An Instructional Medical Video Example
- Concluding remarks

Introduction to the literature

A reference to some of the models is based on the openly available paper:

A. S. Panayides et al., "Position Paper: Artificial Intelligence in Medical Image Analysis: Advances, Clinical Translation, and Emerging Frontiers," *IEEE J. Biomed. Health Inform.*, vol. 10, no. 2, pp. 1187–1202, Feb. 2026, doi: 10.1109/JBHI.2025.3649496.

The basic **learning models** are from:

Torralba, A., Isola, P., & Freeman, W. T. (2024). *Foundations of Computer Vision*. The MIT Press. <https://visionbook.mit.edu/>.

Open Models Library

<https://github.com/pattichis/AIM>

- Open Models for Medical Image Analysis
- Foundation Models
- Instructional Medical Videos
- Generative AI Image + Video Models
- Open Models for Explainability
- PyTorch Video Models, Datasets, and Optimization Resources

If starting projects from zero ...

<https://github.com/pattichis/projects>

- Install Anaconda and Jupyter Notebooks
- Tutorials and Books for Learning AI and Machine Learning
- Project Proposals Guide
- Project assignments
- Final project presentation and deliverables
- Sample projects

Background: Common Architectures for Echocardiography

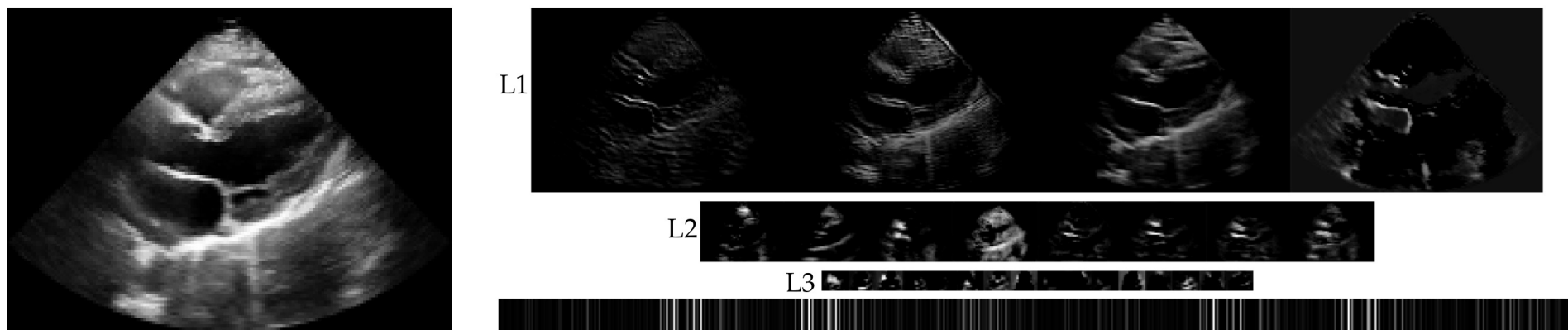
- A large-scale study for all-cause mortality risk.
- Echonet studies

A large-scale multimodal study for predicting mortality risk using minimal and low-parameter models and separable risk assessment



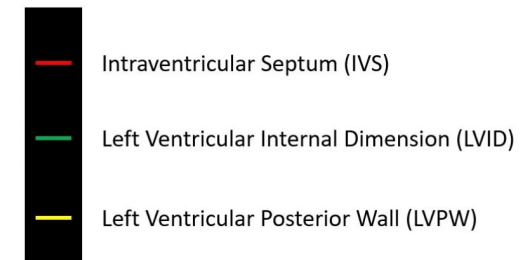
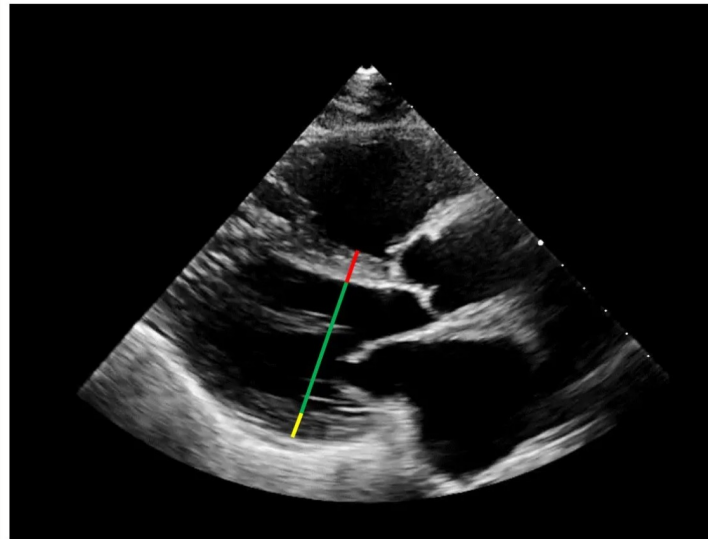
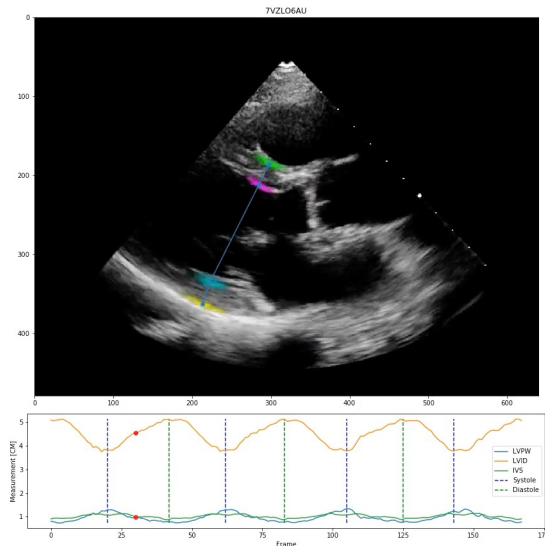
<https://alvarouc.gitlab.io/disiml/index.html>

Visualization of 3D Convolutional Neural Networks



- Video channel visualization through parallel channels
- L1: 4 channels, L2: 8 channels, L3: 16 channels
- Resolution gets halved at every level through max pooling
- Flattening and 50% dropout at the last layer

High-Throughput Precision Phenotyping of Left Ventricular Hypertrophy With Cardiovascular Deep Learning (I of III)

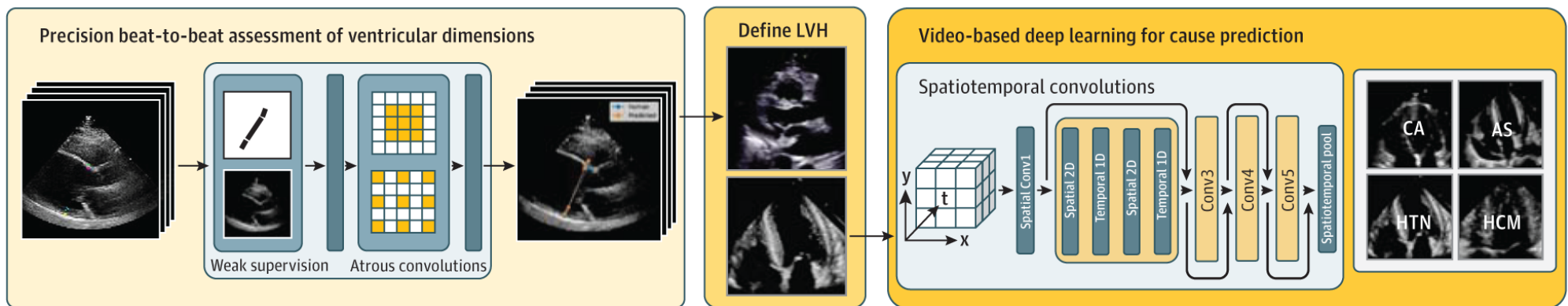


The EchoNet-LVH dataset includes 12,000 labeled echocardiogram videos and human expert annotations (measurements, tracings, and calculations) to provide a baseline to study cardiac chamber size and wall thickness.

Relevant parasternal long-axis (PLAX) and apical 4-chamber 2-dimensional videos, Human clinician annotations of intraventricular septum (IVS), LV internal dimension (LVID), and LV posterior wall (LVPW) measurements were used as training labels to assess ventricular hypertrophy. Parasternal long-axis videos obtained from Stanford Health Care (SHC) were split and used as follows: 9600 for the training set, 1200 for the validation set, and 1200 for the test set. An additional 7767 SHC echocardiogram studies were from patients with defined disease characteristics, including cardiac amyloidosis, HCM, and severe aortic stenosis. From these studies, the apical 4-chamber videos were extracted and used as input data for the hypertrophic disease classification task.

High-Throughput Precision Phenotyping of Left Ventricular Hypertrophy With Cardiovascular Deep Learning (II of III)

A Deep learning workflow



Classifier uses ResNet18 from PyTorch:

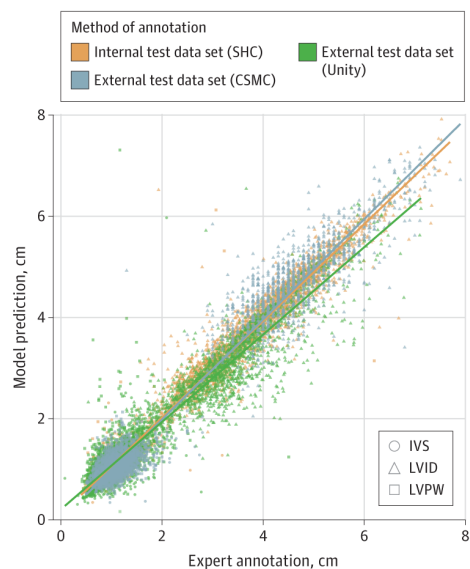
```
self.model = torchvision.models.video.r3d_18()
```

```
self.model.fc = torch.nn.Linear(in_features=512, out_features=1, bias=True)
```

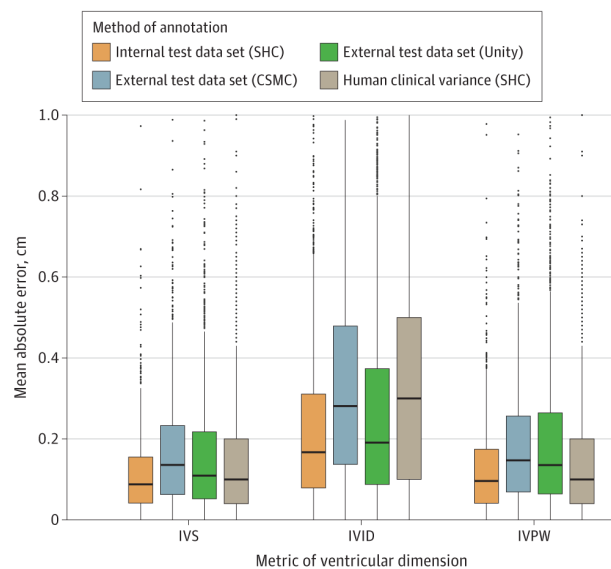
<https://github.com/echonet/lvh>

High-Throughput Precision Phenotyping of Left Ventricular Hypertrophy With Cardiovascular Deep Learning (III of III)

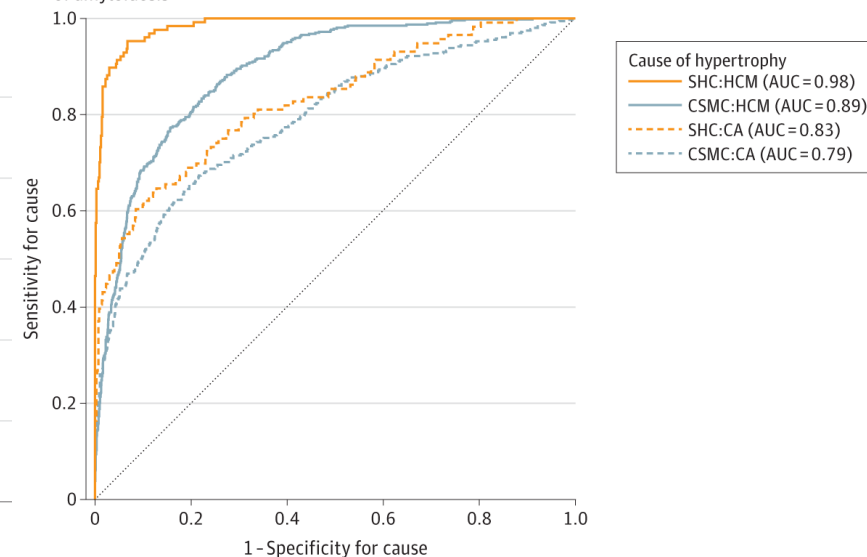
B Correlation of human annotations with model predictions



C Model variations vs human clinical annotation variations



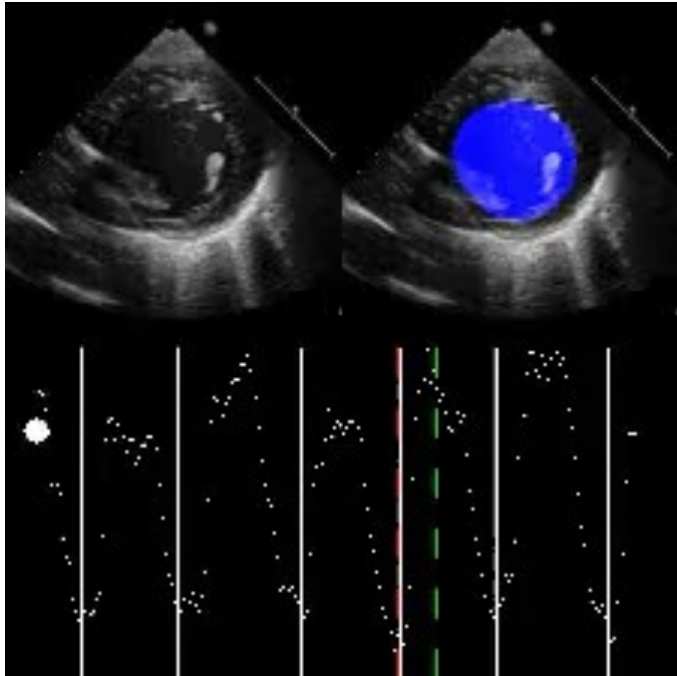
D Receiver operating characteristic curves for diagnosis of amyloidosis



<https://jamanetwork.com/journals/jamacardiology/fullarticle/2789370>

<https://github.com/echonet/lvh>

Video-Based Deep Learning for Automated Assessment of Left Ventricular Ejection Fraction in Pediatric Patients



Results

In both apical 4-chamber and parasternal short-axis views, EchoNet-Peds segments the left ventricle with a Dice similarity coefficient of 0.89.

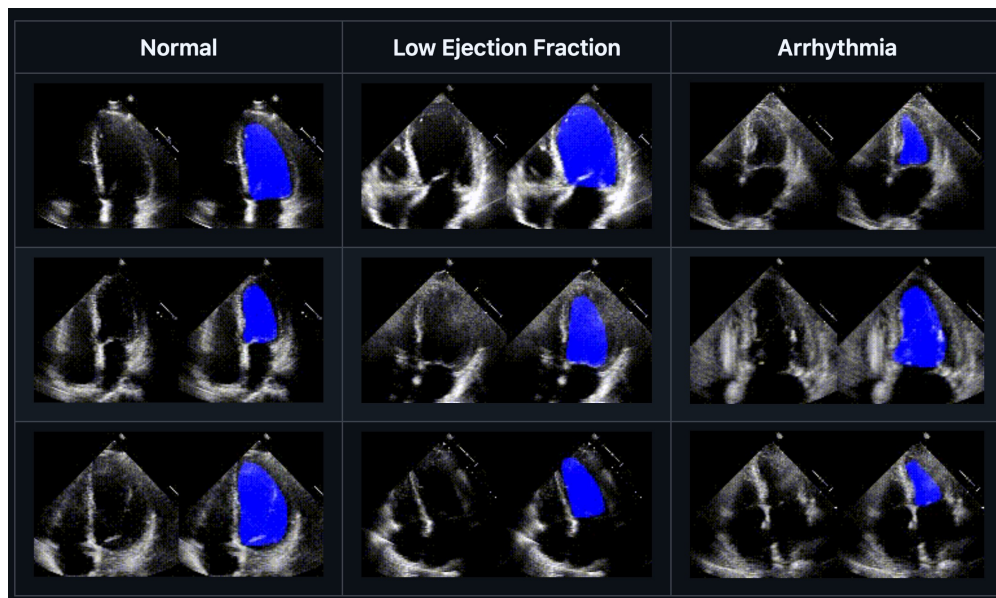
EchoNet-Peds estimates EF with a mean absolute error of 3.66% and can routinely identify pediatric patients with systolic dysfunction (area under the curve of 0.95).

EchoNet-Peds was trained on pediatric echocardiograms and performed significantly better to estimate EF ($P < .001$) than an adult model applied to the same data.

Echocardiogram Videos: 3,176 apical-4-chamber echocardiography videos and 4,424 parasternal short-axis for 2014-2021. Downsampled by cubic interpolation into standardized 112x112 pixel videos.

Measurements: Ejection fraction is expressed as a percentage and is the ratio of left ventricular end systolic volume (ESV) and left ventricular end diastolic volume (EDV) determined by $(EDV - ESV) / EDV$.

Video-based AI for beat-to-beat assessment of cardiac function



Results in real-time:

- Segmentation of LV with Dice of 0.92.
- Ejection fraction with 4.1% MAE.
- Classifies heart failure with AUC of 0.97.
- Better than humans.

External clinic test:

- Ejection fraction with 6.0% MAE.
- Classifies heart failure with AUC of 0.96.

10,030 annotated echocardiogram videos.

Frame-level semantic segmentation of the left ventricle with weakly supervised learning from clinical expert labelling. 3D CNN with residual connections predicts clip-level ejection fraction from the native echocardiogram videos. Segmentations results are combined with clip-level predictions to produce beat-to-beat evaluation of the ejection fraction.

Foundational Material

- Geometric transformations from imaging
- Digital video basics
- Working with limited datasets: basic techniques
- An Introduction to Learning
- Transformer Architectures

Geometric transformations from Imaging

- Rigid body transformations & similarity transformations.
- Affine transformations
- Projective transformations

Hierarchy of Transformations for 3D Inference

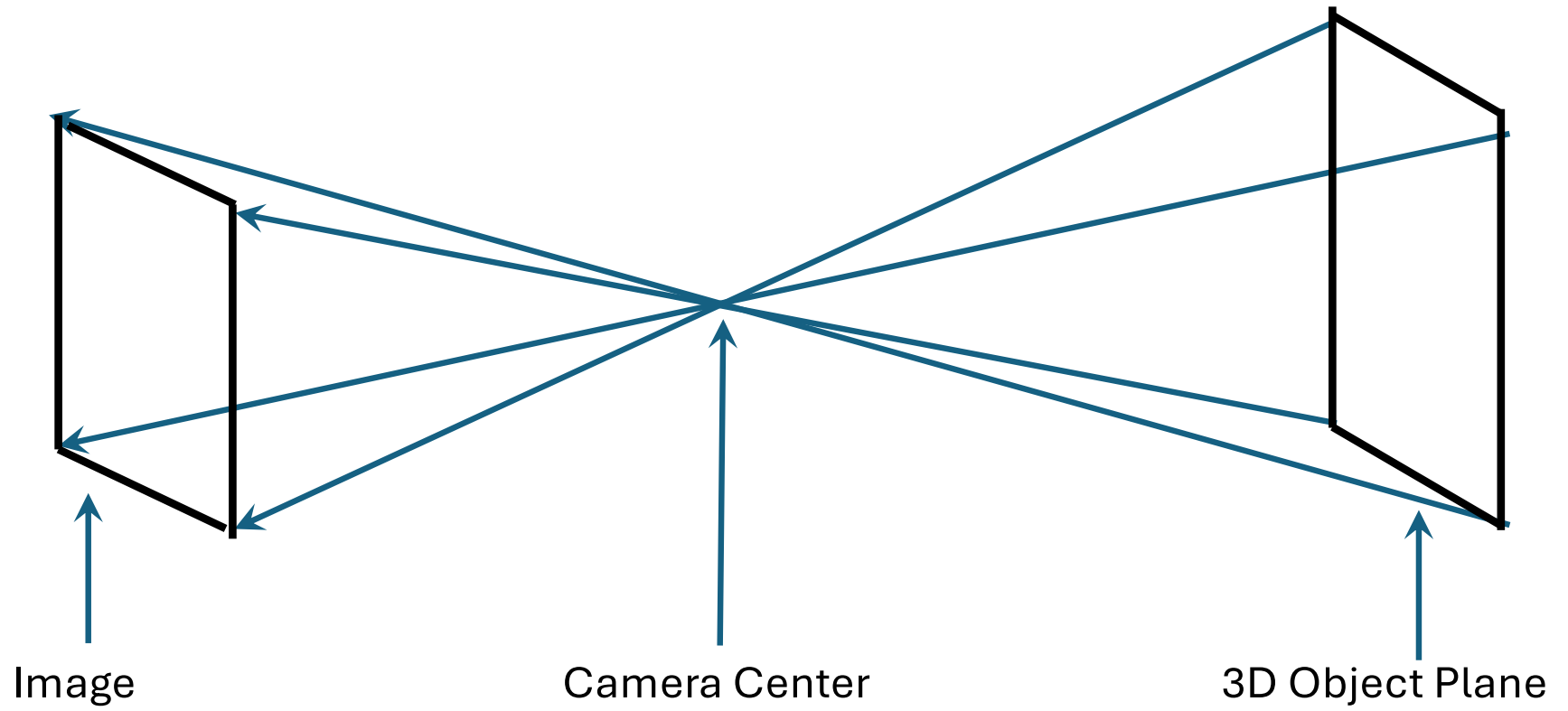
There is a hierarchical relationship among the transformations:

- Rigid body transformations
- Similarity transformations
- Affine transformations
- Projective transformations

Invariants of affine transformations are also invariants of similarity and rigid transformations ...

The opposite is definitely not true.

Uniform Scaling



3D Inference for Rigid Body Transformations

Consider the following operations on the identity map image plane:

- Translations
- Rotations
- Mirror or flip the image
- Any combination of the above

3D Inference without image corrections:

- Lengths
- Areas
- Angles between lines

AtheroRisk: An Integrated Computer Software System for Stroke Risk Stratification Utilizing Carotid Plaque Analysis in Ultrasound Videos

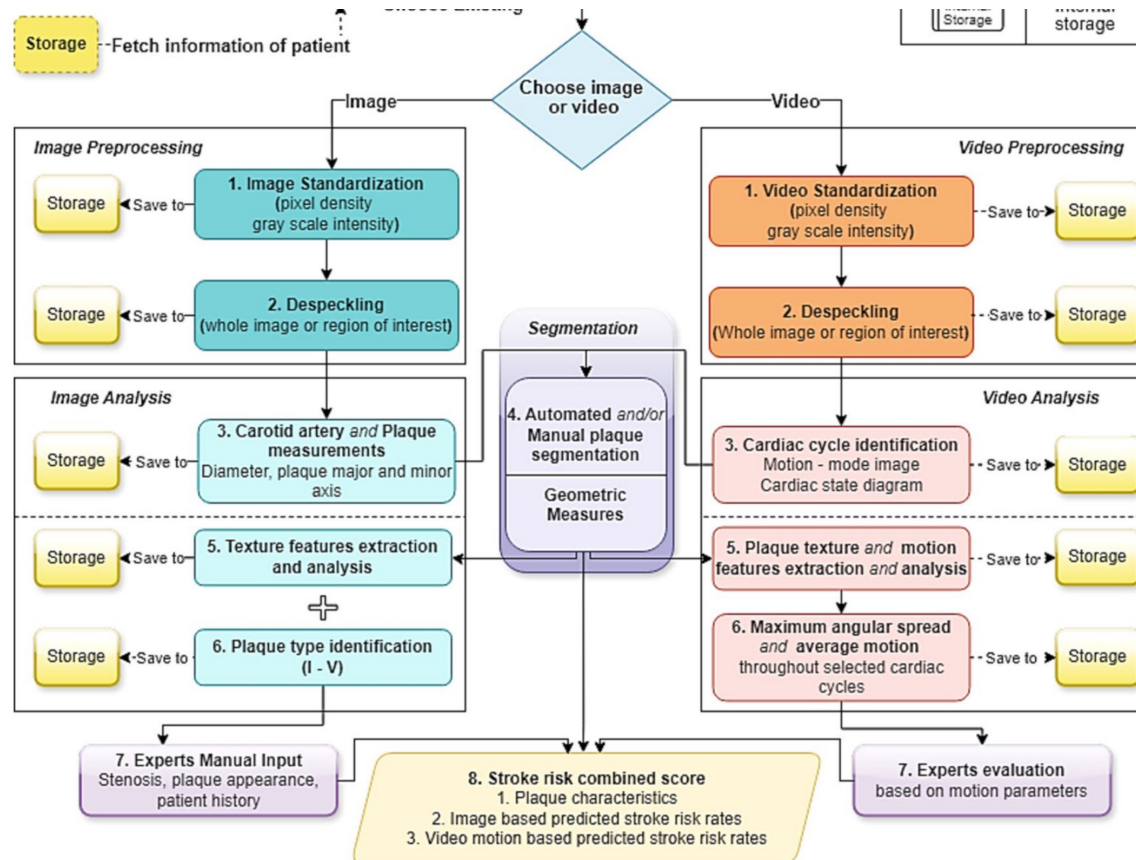


Fig. 2 Flow diagram, demonstrating the available carotid B-mode ultrasound image and/or video preprocessing and analysis pathways for the proposed AtheroRisk software

3D Inference for Similarity Transformations

Consider the following operations on the uniform scaling plane:

- Translations
- Rotations
- Mirror or flip the image
- Uniform re-scaling along both axes
- Any combination of the above

3D Inference **without image corrections:**

- Ratio of lengths
- Ratio of areas
- Angles between lines

3D Inference for Affine Transformations

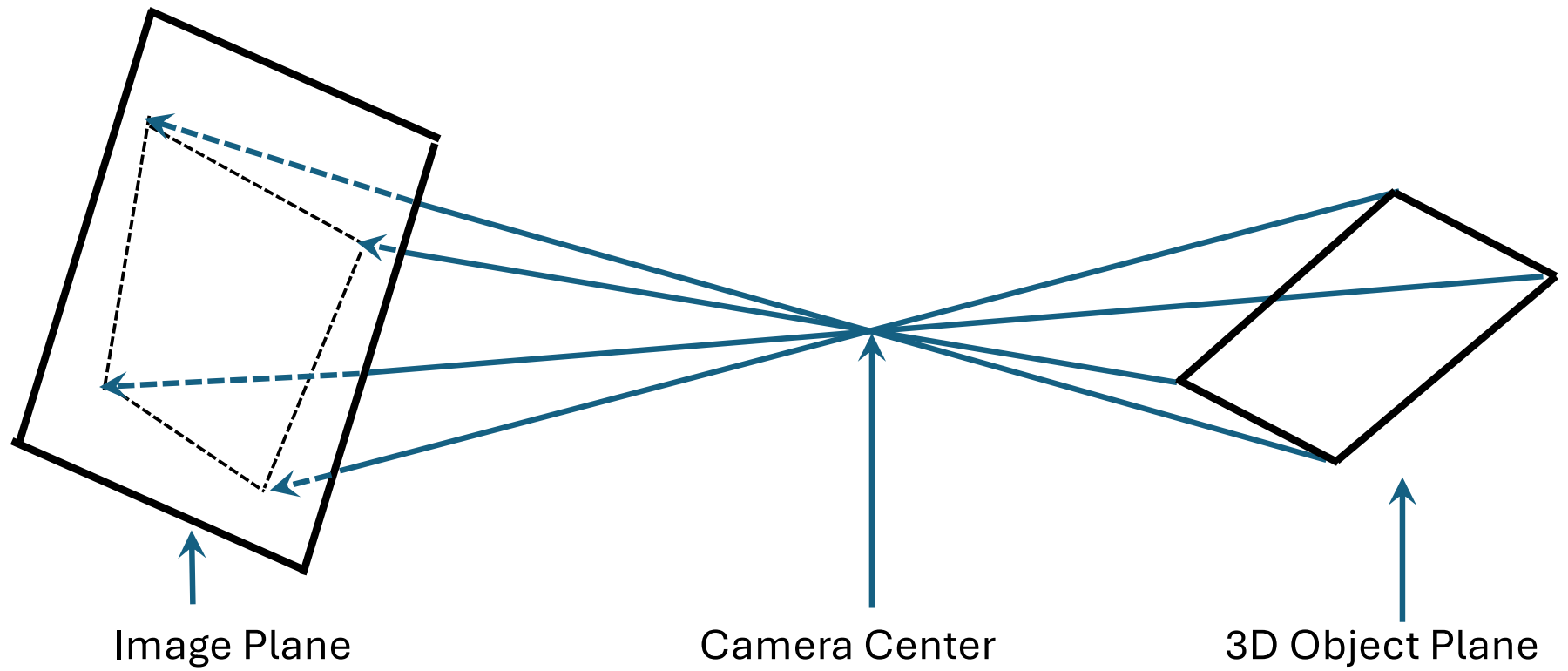
Consider the following operations on the affine plane:

- Any 2D plane transformation described by 2×2 matrices
 - Includes rotations, mirrors, flips, non-uniform re-scalings
- Translations
- Any combination of the above

3D Inference **without image corrections**:

- Parallelism
- Ratio of areas
- **Ratio of lengths on collinear or parallel lines**
- Linear combinations of points (e.g., midpoints, centroids)
- Line at infinity where all lines of the projective plane intersect

General Projective Transformation



3D Inference for Projective Transformations

Any image through the camera center that:

- maps 3D lines to 2D lines on projected plane

3D Inference **without image corrections**:

- Collinearity
- Intersection of lines and concurrency (intersection of 3+ lines)
- **Cross-Ratio or ratio of length ratios**
- Tangency, inflections (point of curvature change: $y=x^3$ at 0)
- Tangent discontinuities and cusps (curve singularity)

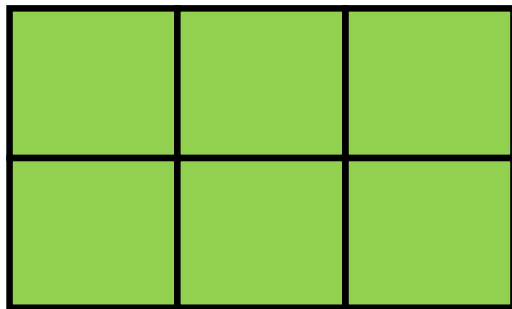
Unless we only look at the features above, we must apply projective transformations!

Digital Video Basics

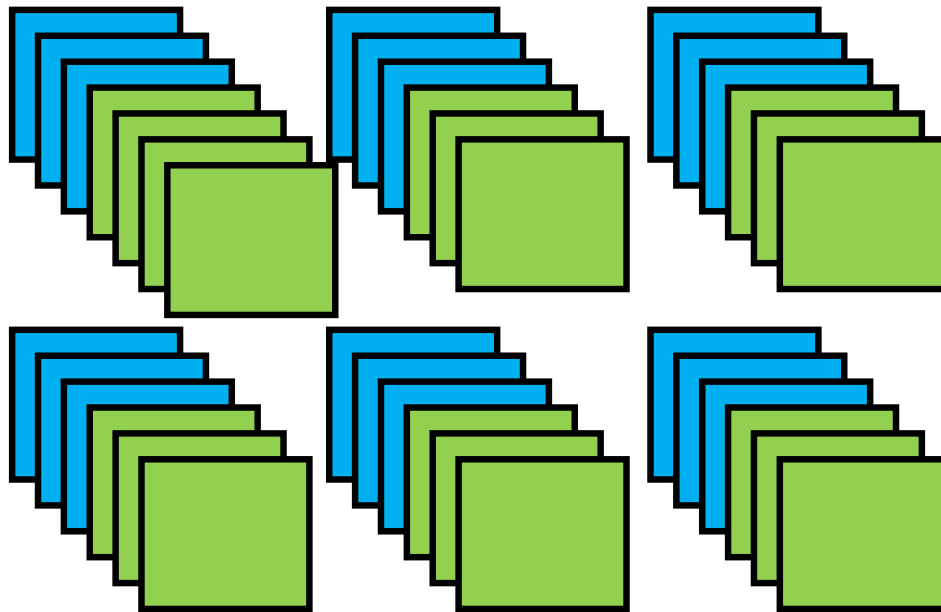
- Video segments
- Video object training
- Dynamic video object recognition
- Parallel processing of video segments
- Examples

Common Inputs to Neural Networks:

Break video into **non-overlapping video segments**

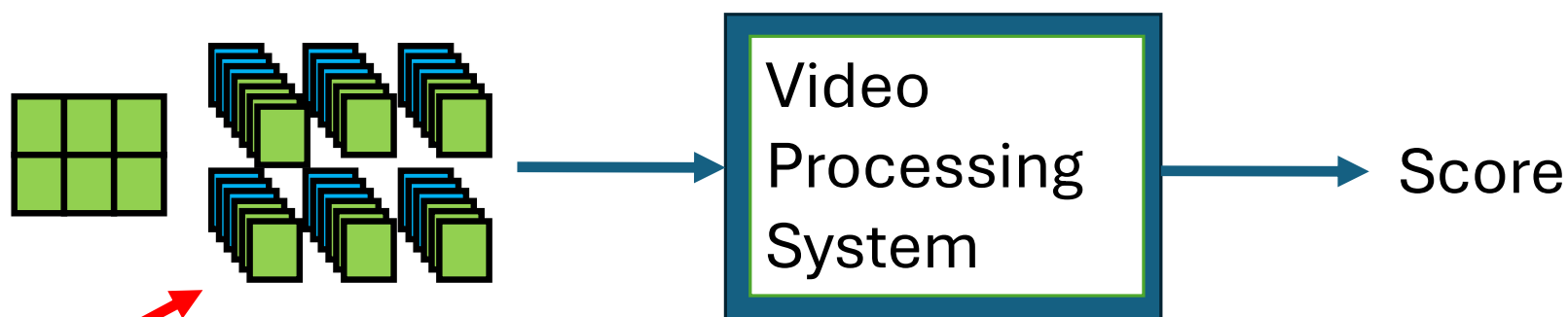


Single video frame

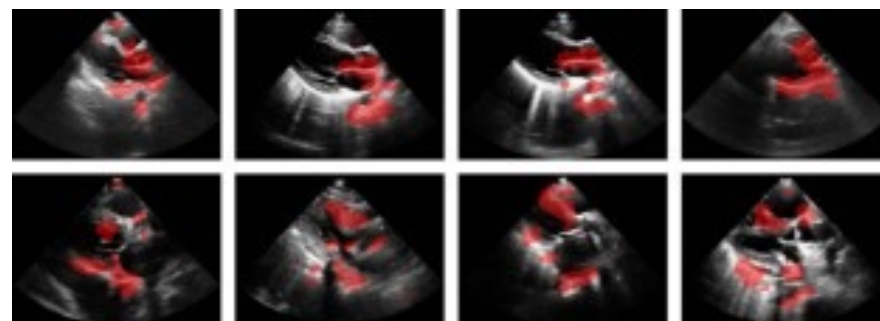


Video segments for processing

Video Segment Importance: Find what your system responds to!



Iteratively zero-out all video segments to find the ones that have the greatest change in score



Ulloa Cerna *et al.*
Deep-learning-assisted analysis of
echocardiographic videos improves predictions of all-cause mortality.
Nat Biomed Eng 5, 546–554 (2021). <https://doi.org/10.1038/s41551-020-00667-9>

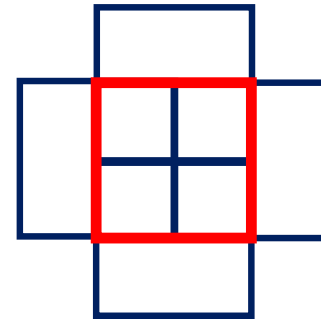
Extending the video segments

Extend with 50% overlapping to avoid cutting structures at the corners:

For each



use



Train for additional scales:



Process video as object detection over sampled frames

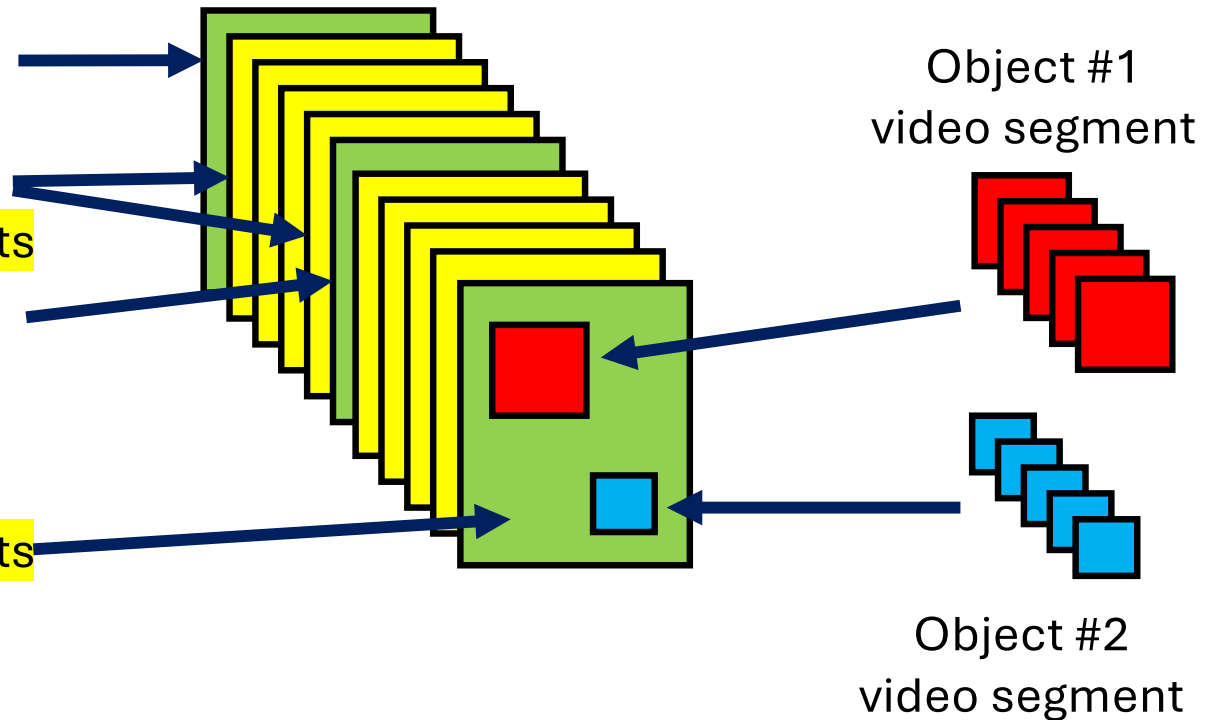
Object detection on this frame

Extract object video segments

Object detection on this frame

Extract object video segments

Object detection on this frame



Example: Hand detection with extracted video segments

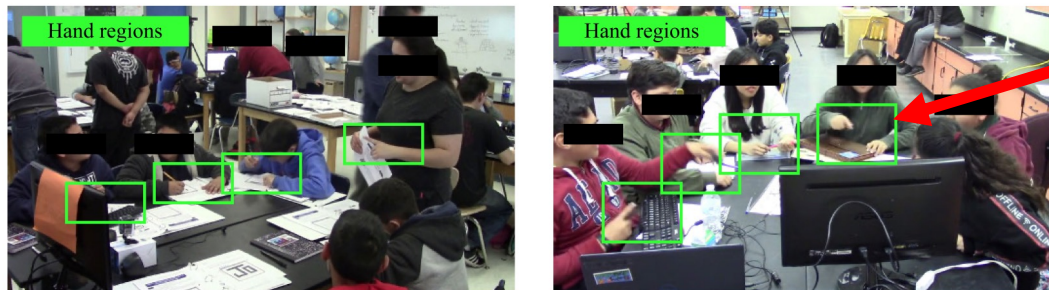
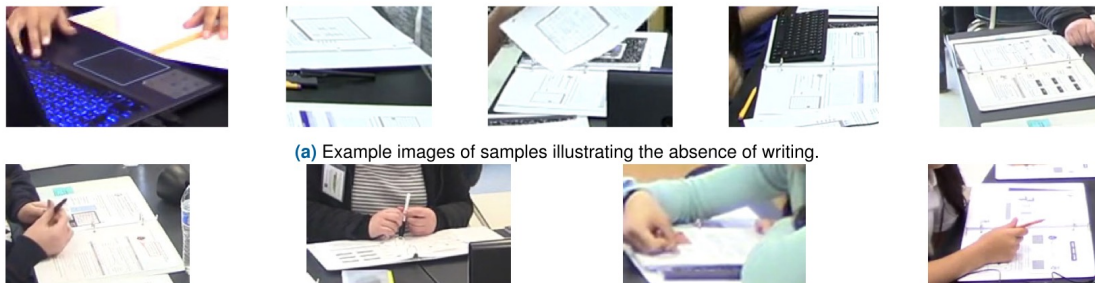


FIGURE 4. AOLME-WD: Sessions used to test our writing detection system. On the left we have the first session (WS1) from group E of cohort 1 (2017) and level 1. On the right we have the second session (TS2) from group C of cohort 3 (2019) and level 1.



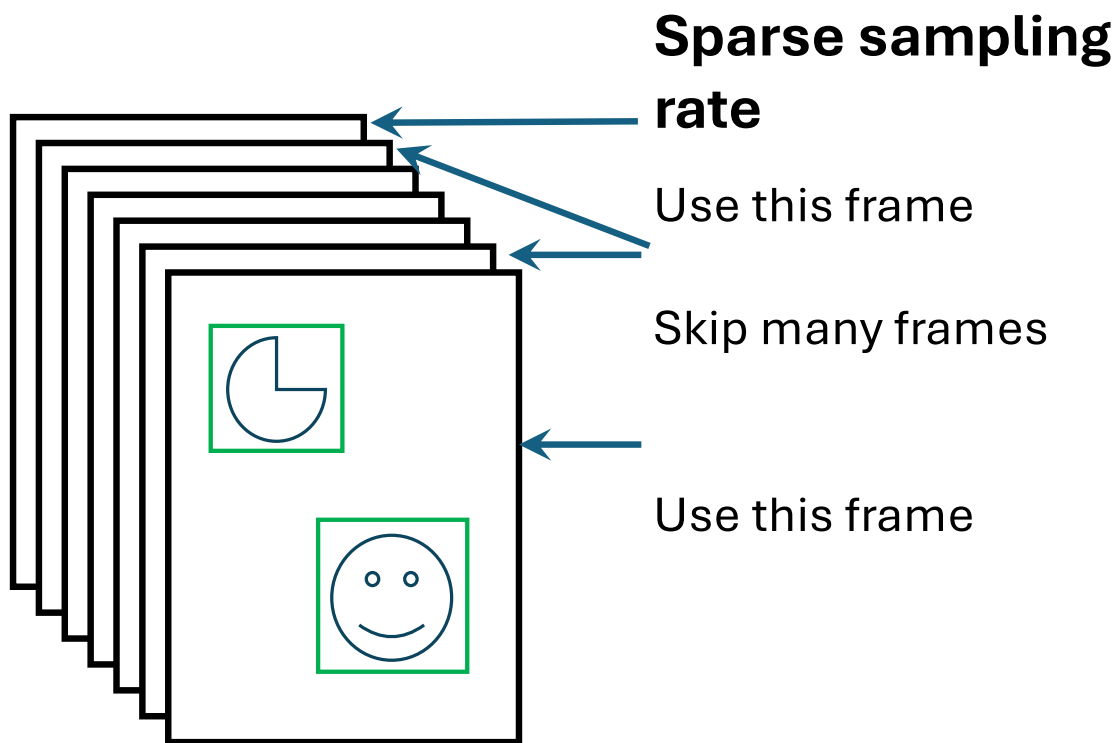
(a) Example images of samples illustrating the absence of writing.

(b) Challenging examples of samples illustrating the absence of writing. As can be observed in the figure, these samples feature hands executing movements similar to those associated with writing.

FIGURE 5. AOLME-WC: No-writing example segments.

V. Jatla et al., "Fast and Accurate Video Analysis and Visualization of Classroom Activities Using Multiobjective Optimization of Extremely Low-Parameter Models," in *IEEE Access*, vol. 13, pp. 81143-81166, 2025, doi: 10.1109/ACCESS.2025.3567437.

Training & Inference: Video object training at **lower frame rate**



Reducing the video frame-rate & number of parameters leads to **Significantly Faster Inference**

TABLE 6. Multiobjective optimization results for VACN architectures in terms of the number of parameters, test-set accuracy, and inference speed. In this table, we report accuracy over the test-set. This accuracy increased with the test-set AUC. Inference speed is controlled in terms of the frame rate and the number of parameters. FPS refers to the number of frames per second of the input video. Thus, at 10 FPS, the network is applied to the video sampled at 10 frames per second. The number of parameters is controlled in terms of the number of dyads. We choose the highlighted configuration as the most balanced configuration (see text).

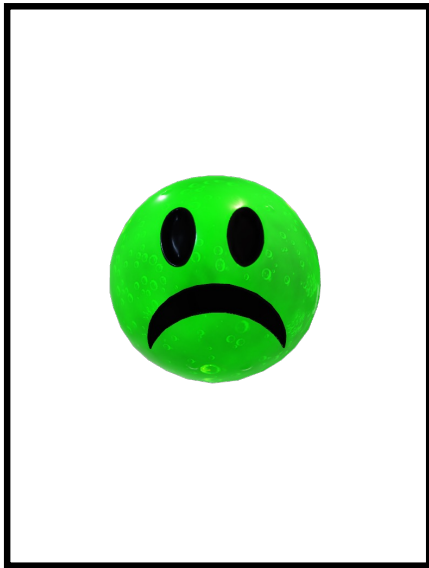
Num. of dyads	FPS	Num. Of Params	Typing Acc.	Writing Acc.
1	10	657K	53.33	39.93
2	10	47K	61.25	39.93
3	10	7.8K	61.25	57.34
4	10	18.7 K	69.59	63.09
1	20	657K	53.33	38.90
2	20	47K	53.33	61.09
3	20	7.8K	62.08	59.87
4	20	18.7K	65.83	63.22
1	30	657K	53.33	39.93
2	30	47K	53.33	39.93
3	30	7.8K	62.91	61.54
4	30	18.7K	67.91	64.05

TABLE 8. Comparative results for video activity classification in terms of the number of parameters, inference speed, and memory requirements. The model performance is measured using AUC and accuracy. We measure inference speed based on the original video playback speed (playback speed = 1×). The best performance is highlighted against a green background. The proposed approach uses over a 1000 times less parameters, requires far less memory, runs faster, and performs better than all other methods.

Method	# Param.	Inf. speed	GPU Mem. in MB	Test AUC	Test acc.	Detection AUC on complete sessions
Classification on AOLME-TC dataset						AOLME-TD sessions
I3D	27.2M (1437×)	3×	5051 (20×)	0.66	64.58	0.75
Slowfast	33.5M (1787×)	3×	6318 (25×)	0.71	61.25	0.48
TSM	23.5M (1252×)	118×	6971 (28×)	0.59	58.75	0.77
TSN	23.5M (1252×)	4×	5593 (23×)	0.74	65	0.76
Ours-opt	18.7K (1×)	154×	245 (1×)	0.76	69.58	0.83
Classification on AOLME-WC dataset						AOLME-WD sessions
I3D	27.2M (1437×)	3×	5051 (20×)	0.66	59.58	0.56
Slowfast	33.5M (1787×)	3×	6318 (25×)	0.57	53.67	0.48
TSM	23.5M (1252×)	118×	6971 (28×)	0.50	47.60	0.41
TSN	23.5M (1252×)	4×	5593 (23×)	0.62	61.66	0.45
Ours-opt	18.7K (1×)	154×	245 (1×)	0.67	63.09	0.59

V. Jatla et al., "Fast and Accurate Video Analysis and Visualization of Classroom Activities Using Multiobjective Optimization of Extremely Low-Parameter Models," in *IEEE Access*, vol. 13, pp. 81143-81166, 2025, doi: 10.1109/ACCESS.2025.3567437.

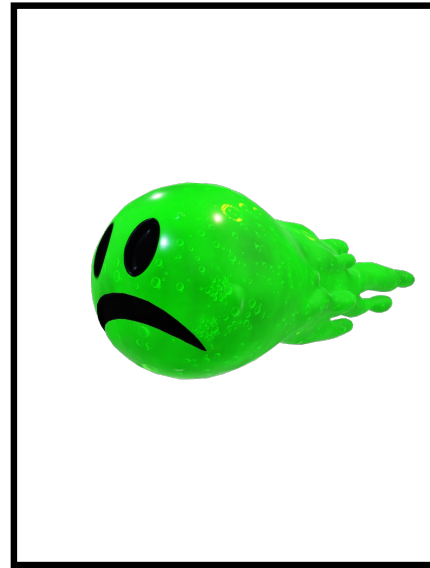
3D video object pose recognition challenges



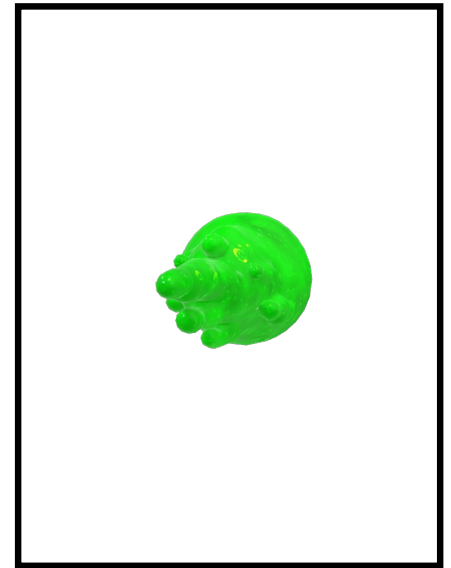
Front



Transformations

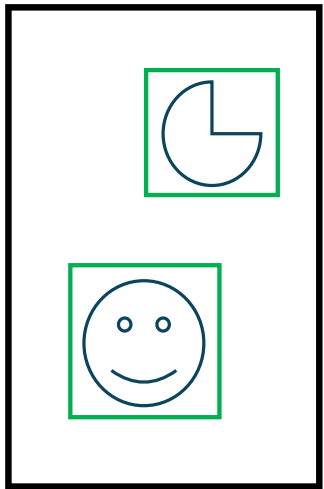


Side

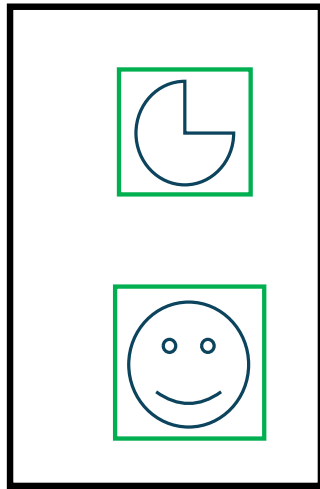


Back

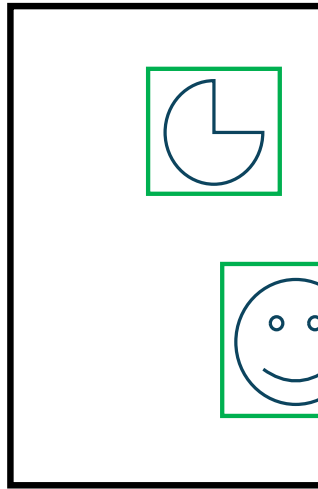
Translational Video motions



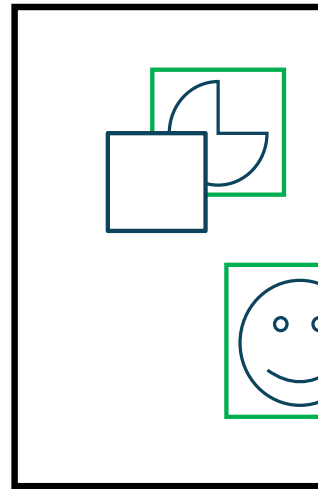
Initial
objects



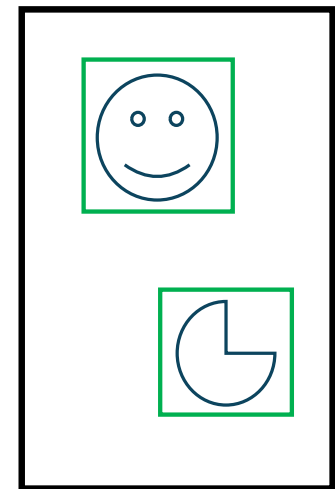
Minor
Movements



Moving out

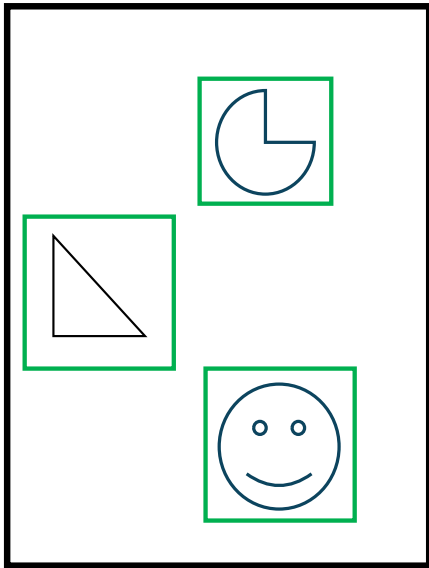


Occlusion

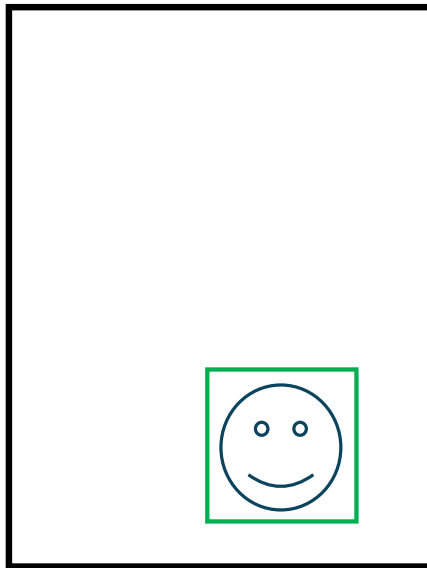


Rearrangements

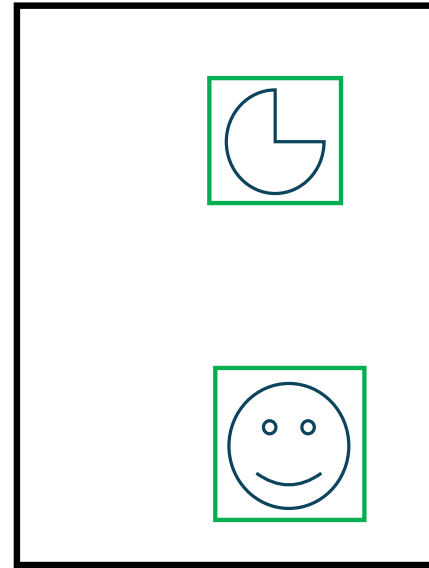
Dynamic video object recognition challenges



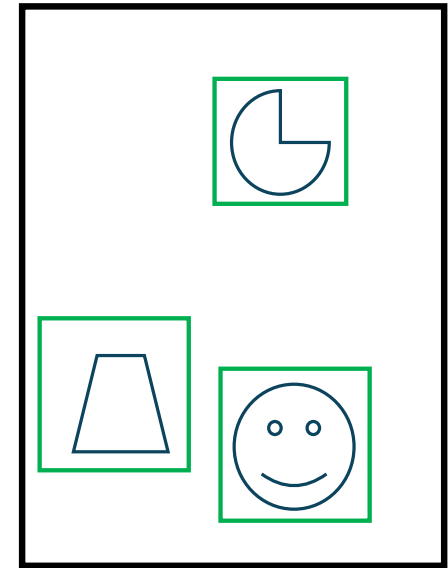
Initialization
object
detection



Disappearing
objects

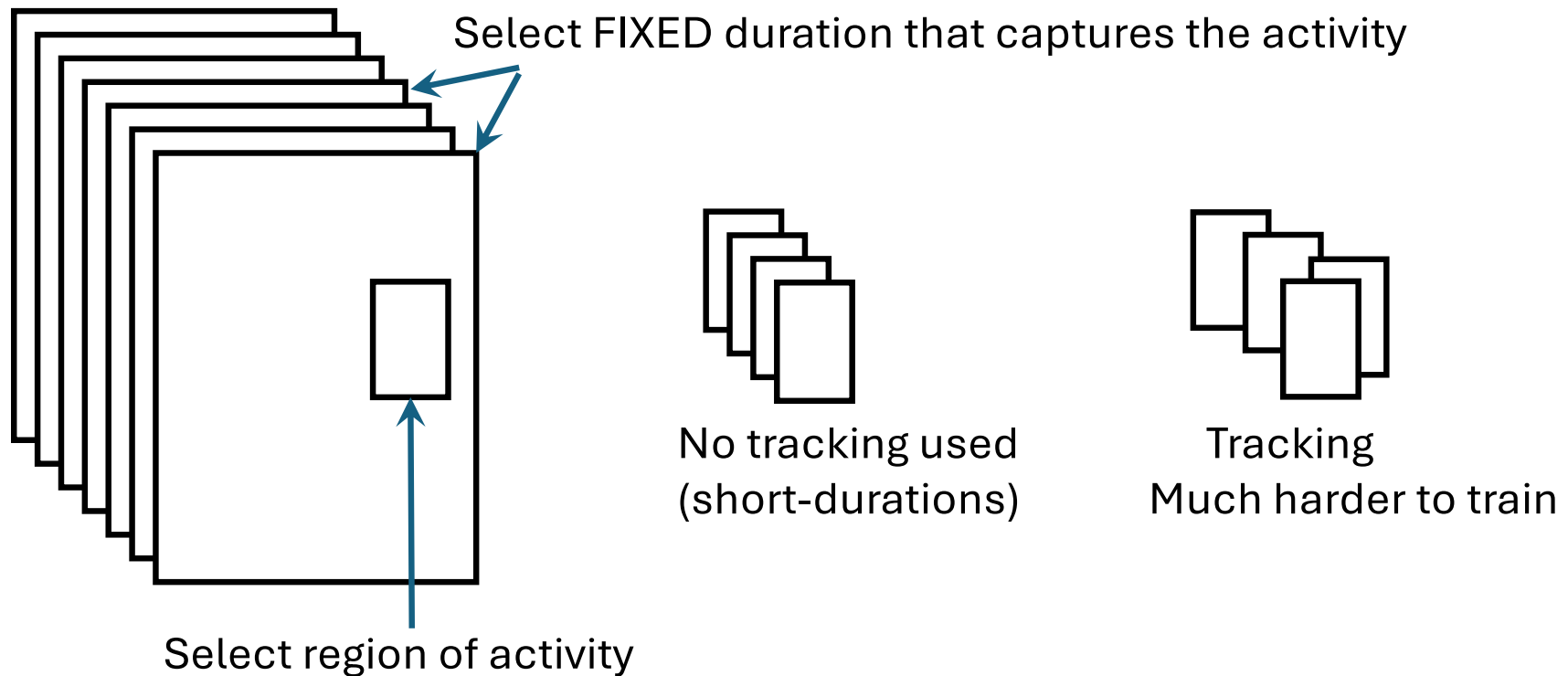


Reappearing
object

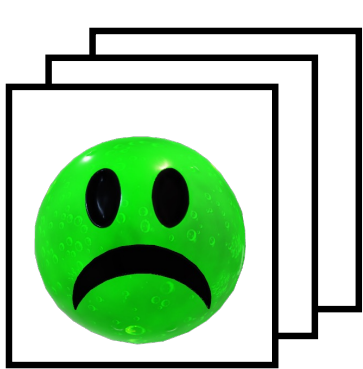


New
object

Training: Select short video segments and frame rate



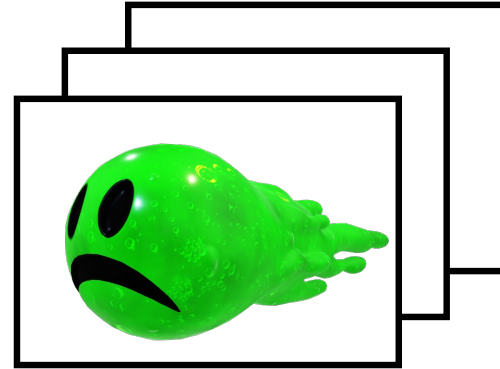
Training: Video activities from different poses



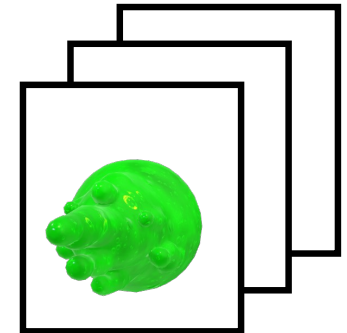
Front-facing
activity



Rotated
activity



Side-facing
activity

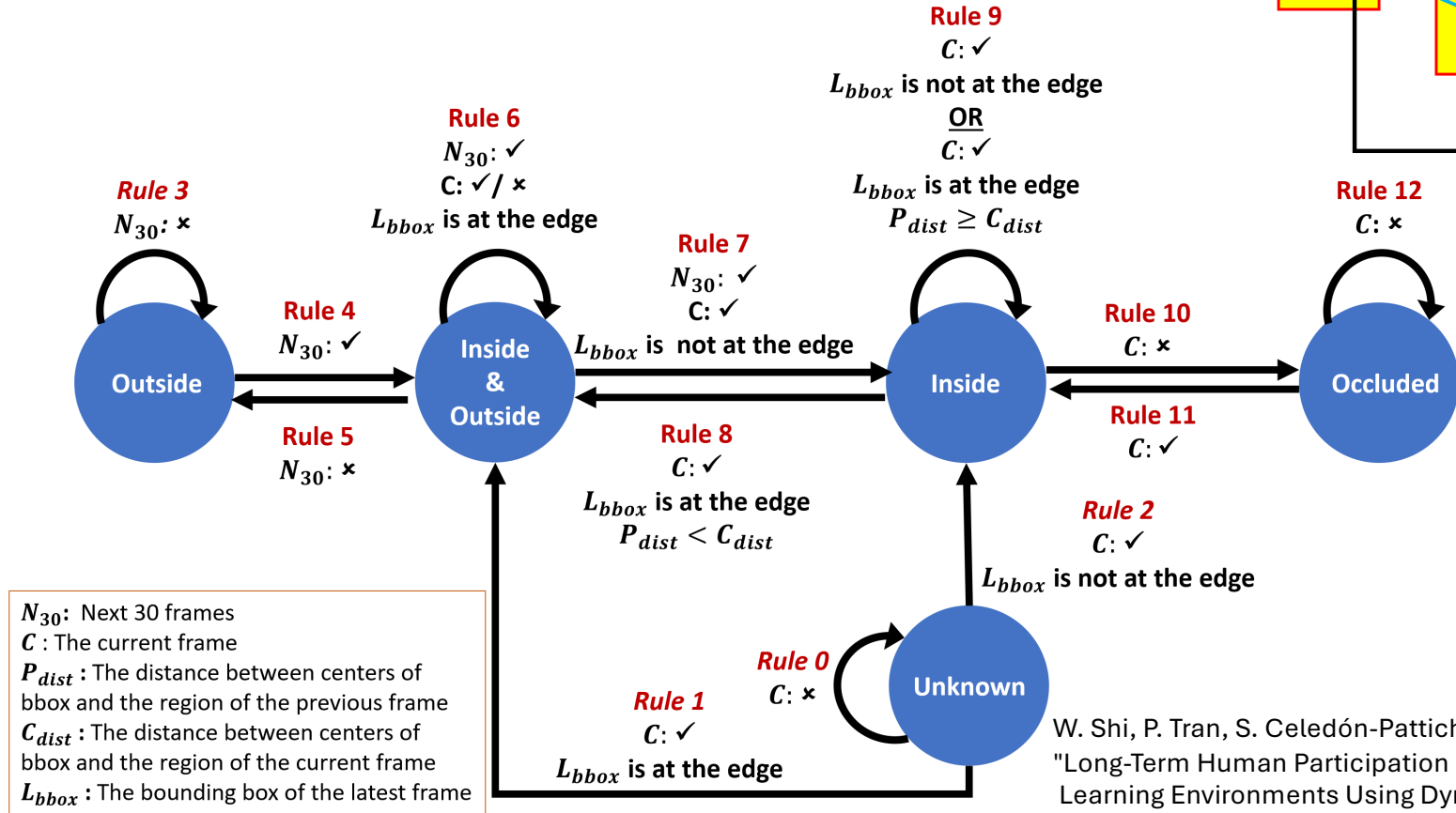
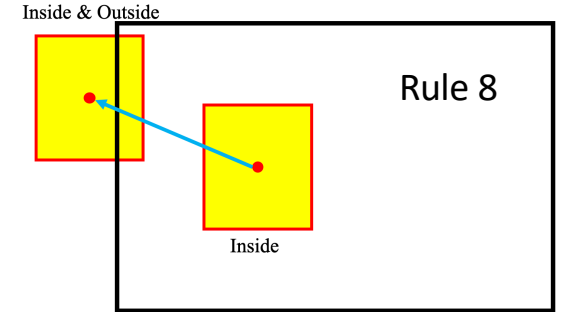


Back-facing
activity

Critical Decisions when working with Video Segments

- What is the **duration** of the video segment?
 - Duration should contain the activity (a few seconds)
- Do you need to **reduce the frame rate**?
 - 30 frames per second maybe too dense to capture the movement.
 - People's faces may only need to be processed at 1 frame per second.
- Do you need to reduce the **video frame resolution**?
 - You may need to resize the video frames for pre-trained image classifiers.
 - Reduce for faster processing (e.g., 4k to 1k)
 - Objects and motion should be clearly visible in reduced resolution
- **Transformations, multiple views and occlusions**?
 - Transformations can be used to capture variations from the **same view**.
 - Consider a variation of different viewing angles that show **different components**.
 - Develop a strategy for dealing with **occlusions**.

Dynamic Participant Tracking

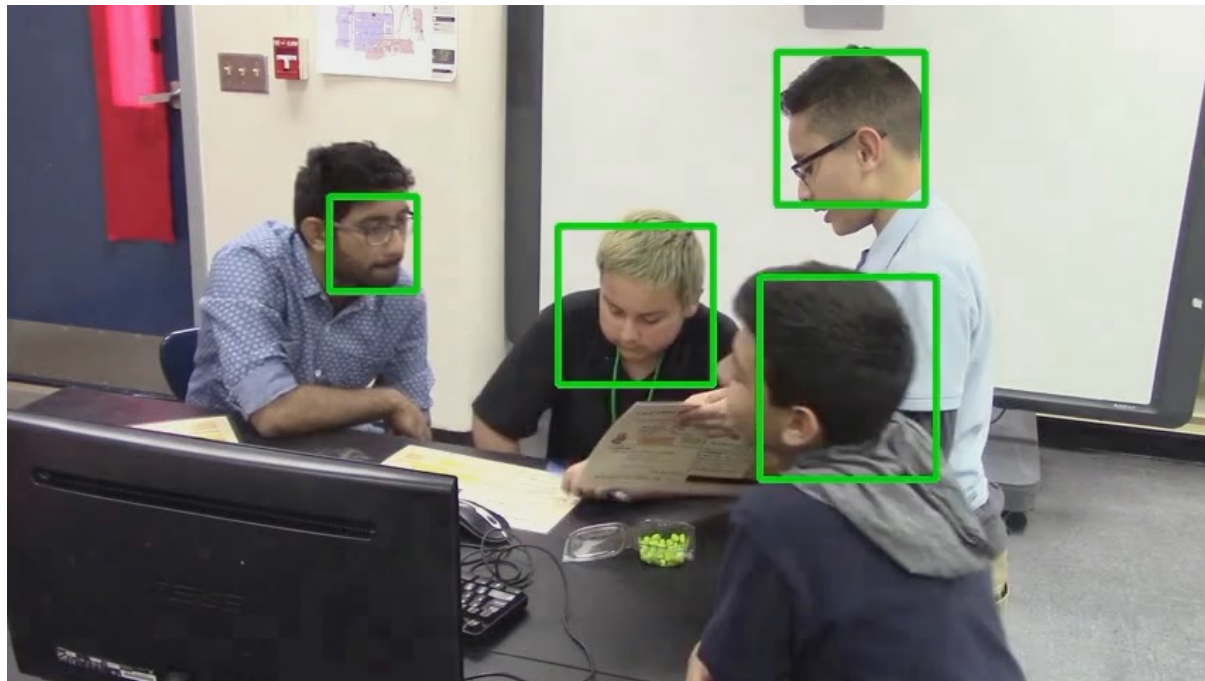


N_{30} : Next 30 frames
 C : The current frame
 P_{dist} : The distance between centers of
 bbox and the region of the previous frame
 C_{dist} : The distance between centers of
 bbox and the region of the current frame
 L_{bbox} : The bounding box of the latest frame

W. Shi, P. Tran, S. Celedón-Pattichis and M. S. Pattichis,
 "Long-Term Human Participation Assessment in Collaborative
 Learning Environments Using Dynamic Scene Analysis,"
 in *IEEE Access*, vol. 12, pp. 53141-53157, 2024.



Dynamic Participant Tracking



Perfect detection

Green = 'Inside', **Red** = 'Occluded'

Real-time video processing delay considerations

Video segment places a minimum bound:

$$\text{Minimum delay} = 1 / \text{frames-per-second} * \text{Number-of-video-frames}$$

Actual delay is even slower:

$$\begin{aligned} \text{Actual delay} = & \text{video partition time} \\ & + \text{video segments processing time} \\ & + \text{collect results back into the video} \end{aligned}$$

If processing all video segments in parallel, then:

$$\begin{aligned} \text{video segments processing time} = & \text{launch parallel threads time} \\ & + \text{single video segment processing time} \\ & + \text{collect results back into the video} \end{aligned}$$

Working with limited datasets: Basic Techniques

- Data augmentation
- Adversarial training
- Active learning

We need to transform image and video datasets:

Data Augmentation and Other Applications

Neural network features are not invariant!

We need the following **single-view transformations** (**no new data needed**):

- Projective transformations (including multiple scales and overlaps)
- Color image intensity variations.
- Adapt frame rates.
- Occlusions

Multiple-view transformations (**requires new data**):

- Collect examples from angles that expose **different structures**.
- Repeat single-view transformations.

Implement in dataloader without the need to save the datasets.

Use of generative models covered later!

Adversarial Training: Improve the robustness of your Neural Net

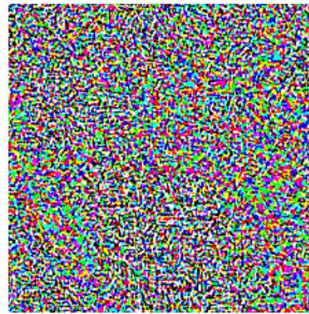


x

“panda”

57.7% confidence

$+ .007 \times$



$\text{sign}(\nabla_x J(\theta, x, y))$

“nematode”

8.2% confidence

$=$



$x +$

$\epsilon \text{sign}(\nabla_x J(\theta, x, y))$

“gibbon”

99.3 % confidence

Adversarial Attack:

Corrupt the input image to force the classifier to an error.

Retrain the classifier to correctly classify the corrupted images.

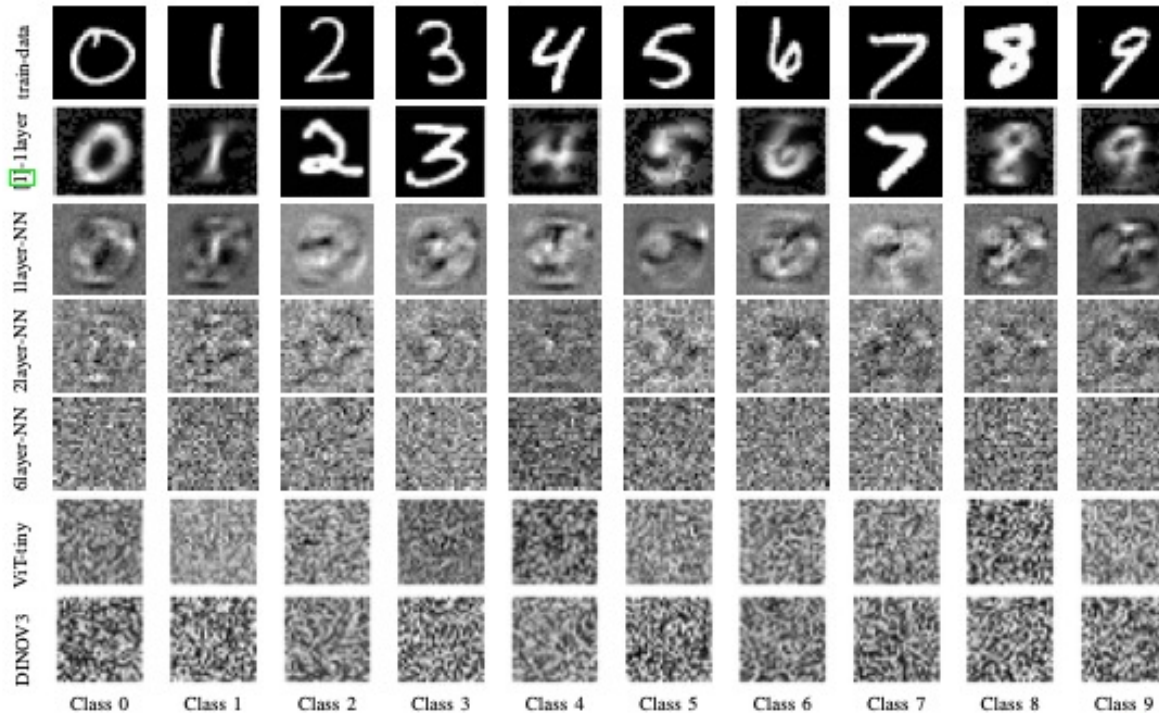
https://docs.pytorch.org/tutorials/beginner/fgsm_tutorial.html

In the context of adversarial robustness, one form of [Equation 36.1](#) is to set T to be an epsilon-ball attack, that is, $T(\mathbf{x}) = \mathbf{x} + \epsilon$:

$$\arg \min_f \mathbb{E}_{\mathbf{x}, \mathbf{y}} \left[\max_{\epsilon \text{ s.t. } \|\epsilon\| < r} [\mathcal{L}(f(\mathbf{x} + \epsilon), \mathbf{y})] \right] \quad (36.2)$$

<https://visionbook.mit.edu/>

A fundamental limit of Neural Nets!



You can attack Neural Networks and get near-perfect classification results with null-space noise!

Each column gives the same output with ≥ 0.9 probability.

Fig. 3. Generated ideal input images using different algorithms. **train-data row**: reference images given in the MNIST training data for each class. **1-layer row**: ideal images generated in **1-layerNN**, **2layerNN**, and **6layer NN** rows: ideal images generated BACKPASSINV algorithm on FCNNs, with null-space noise added ($\text{std} = 0.1$). **ViT-tiny** and **DINOv3** rows: generated images using FORWARDPASSINV(.). All generated images from new methods produce a probability score ≥ 0.9 for each digit class and ≤ 0.1 for the other ones.

[Rebecca Pattichis](#), [Sebastian Janampa](#), [Constantinos S. Pattichis](#), [Marios S. Pattichis](#),
“Inverting Neural Networks: New Methods to Generate Neural Network Inputs from Prescribed Outputs”
2026 IEEE Southwest Symposium on Image Analysis and Interpretation (SSIAI).

Active Learning:

Select a **smaller number of samples** for ground truth

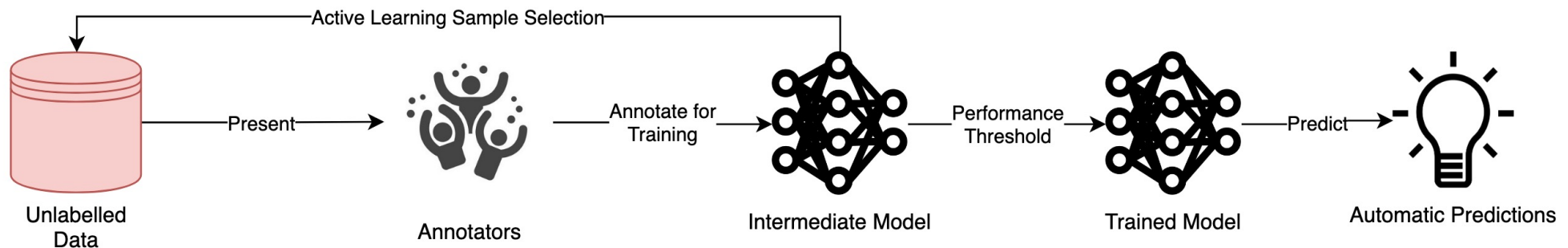


Fig. 1. Overview of Active Learning frameworks.

A survey on active learning and human-in-the-loop deep learning for medical image analysis
Medical Image Analysis, vol. 71, July 2021.

Active Learning: How to Select Informative Samples

Select samples whose labels are uncertain based on:

- Uncertainty sampling based on entropy (over the classes) or error reduction (over the classes)
- Disagreement between classifiers
- Select samples based on adversarial learning
 - Perturb correctly classified sample to give the wrong output
- Select labels within and across video using cluster informativeness and diversity [1]
- Apply active learning methods for automatically labeling the dataset [2]
- Apply frame selection based on uncertainty and diversity [3]

[1] Rana, Aayush J., and Yogesh S. Rawat. "Hybrid Active Learning via Deep Clustering for Video Action Detection." In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 18867-18877. 2023.

[2] Yang, Jie. "Automatically labeling video data using multi-class active learning." In *Proceedings Ninth IEEE international conference on computer vision*, pp. 516-523. IEEE, 2003.

[3] Goswami, Debanjan, and Shayok Chakraborty. "Active Learning for Video Classification with Frame Level Queries." In *2023 International Joint Conference on Neural Networks (IJCNN)*, pp. 1-9. IEEE, 2023.

An Introduction to Learning

- Empirical Risk Minimization (ERM)
- Finetuning
- Building larger models, Knowledge distillation, Learning pre-text, tasks
- Contrastive learning
- Learn a prompt or a token

9.5 Empirical Risk Minimization: A Formalization of Learning from Examples

The three ingredients from the last section can be formalized using the framework of (ERM). This framework applies specifically to the supervised setting where we are learning a function that predicts $\mathbf{y}$ from $\mathbf{x}$ given many training examples $\{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\}_{i=1}^N$. The idea is to minimize the average error (i.e., risk) we incur over all the training data (i.e., empirical distribution). The ERM problem is stated as follows:

$$\arg \min_{f \in \mathcal{F}} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f(\mathbf{x}^{(i)}), \mathbf{y}^{(i)}) \quad \triangleleft \quad \text{ERM}$$

Here, $\mathcal{F}$ is the hypothesis space, $\mathcal{L}$ is the loss function, and $\{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\}_{i=1}^N$ is the training data (example { **input**, **output** } pairs), and f is the learned function.

<https://visionbook.mit.edu/>

Transfer Learning: An ERM Summary

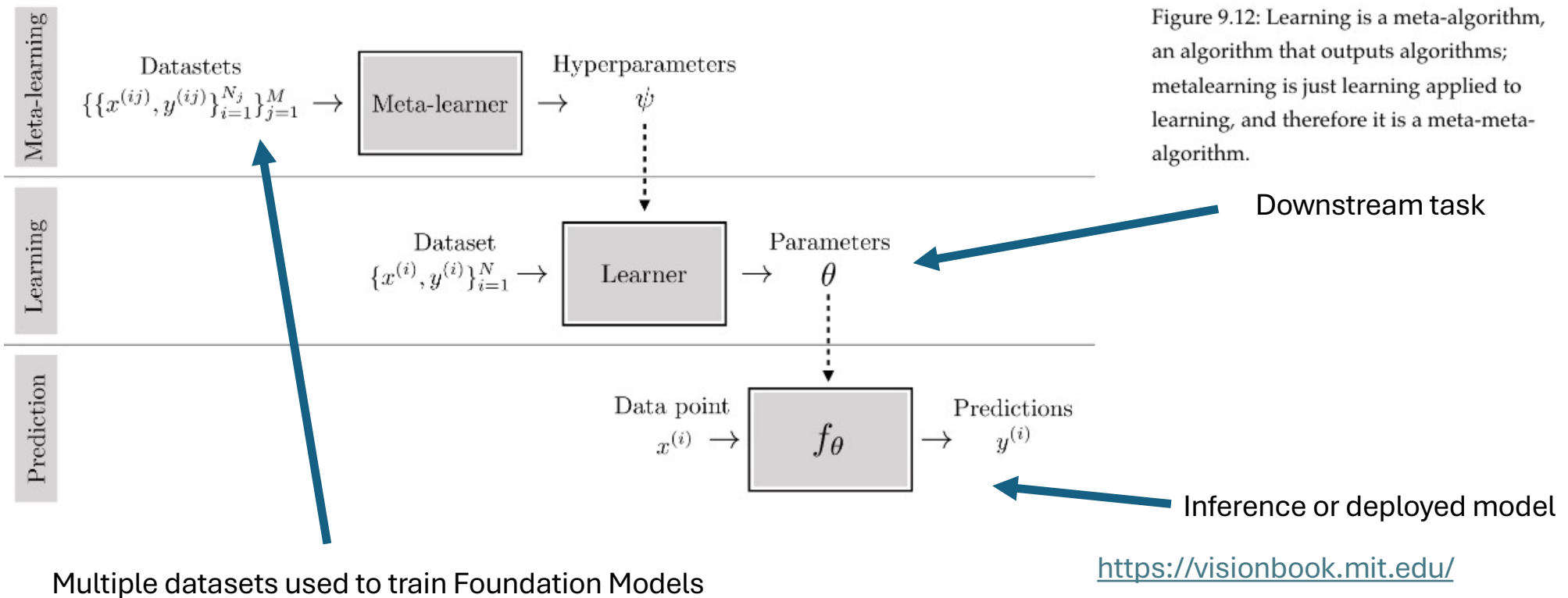
In table [Table 32.1](#), we present the relationship these methods by framing them all as empirical risk minimization with different components optimized during pretraining and/or during adaptation.

Method	Learning	Inference
Generative data	$\arg \min_{\theta} \mathbb{E}_{x,y} [\mathcal{L}(f_{\theta}(x), y)]$	$f_{\theta}(x)$
Distillation	$\arg \min_{\theta} \mathbb{E}_x [\mathcal{L}(f_{\theta}(x), t(x))]$	$f_{\theta}(x)$
Prompting	$\arg \min_{\epsilon} \mathbb{E}_{x,y} [\mathcal{L}(f_{\theta}(x \oplus \epsilon), y)]$	$f_{\theta}(x \oplus \epsilon)$
Finetuning	$\arg \min_{\theta} \mathbb{E}_{x,y} [\mathcal{L}(f_{\theta}(x), y)]$	$f_{\theta}(x)$
Probes / heads	$\arg \min_{\phi} \mathbb{E}_{x,y} [\mathcal{L}(h_{\phi}(f_{\theta}(x)), y)]$	$h_{\phi}(f_{\theta}(x))$
Learned loss	$\arg \min_{\theta} \mathbb{E}_{x,y} [\mathcal{L}(f_{\theta}(x), y)]$	$f_{\theta}(x)$
Learned optimizer	$\arg \min_{\theta} \mathbb{E}_{x,y} [\mathcal{L}(f_{\theta}(x), y)]$	$f_{\theta}(x)$

Table 37.1: Different kinds of transfer learning presented as empirical risk minimization (ERM). *Notes:* The $\oplus$ symbol indicates concatenation or addition (or other kinds of signal combination). Domain adaptation does not map neatly to ERM so we omit it here.

<https://visionbook.mit.edu/>

Learning and Meta-Learning (Learning to Learn) with my Annotations



Adapt a pretrained model to a new dataset: Finetuning

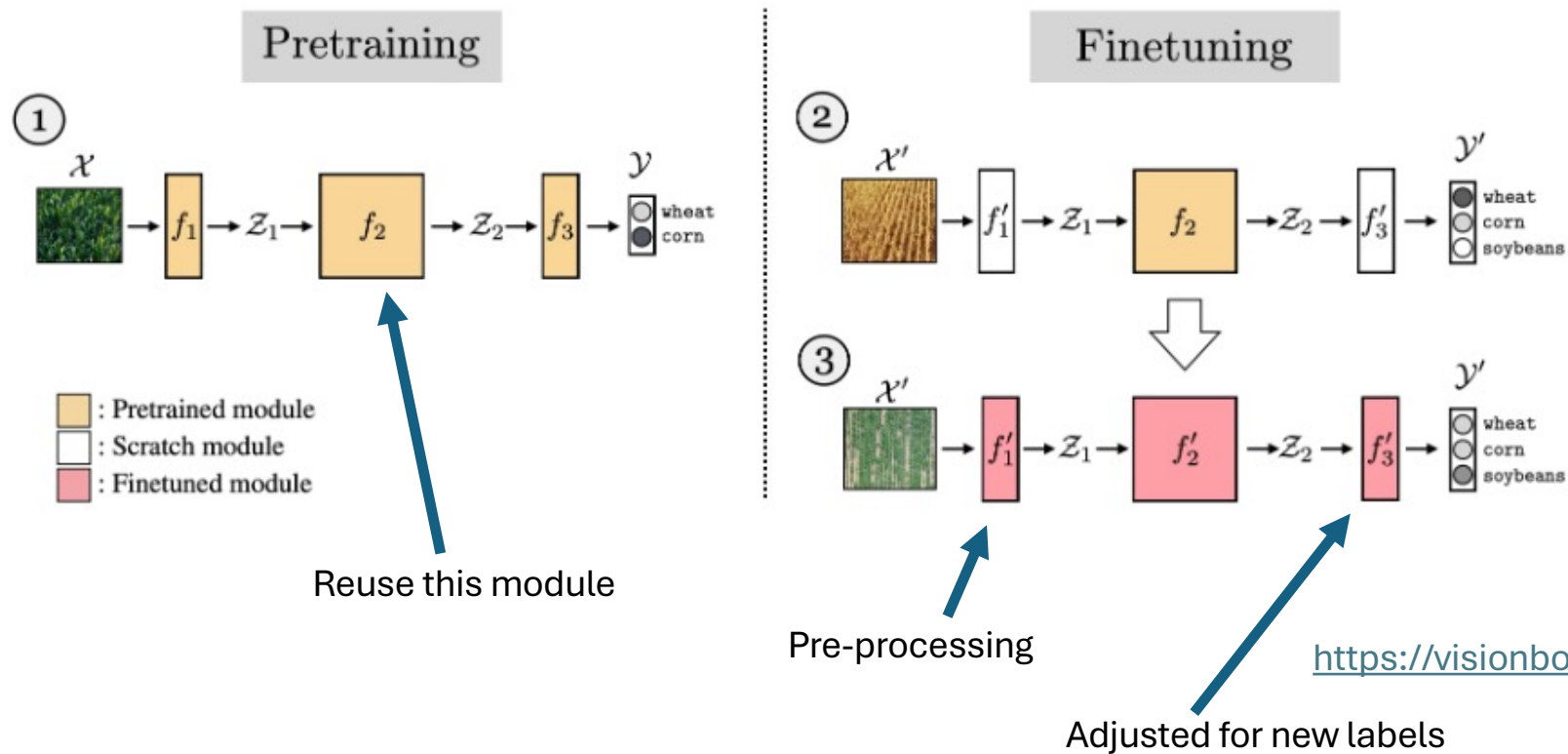
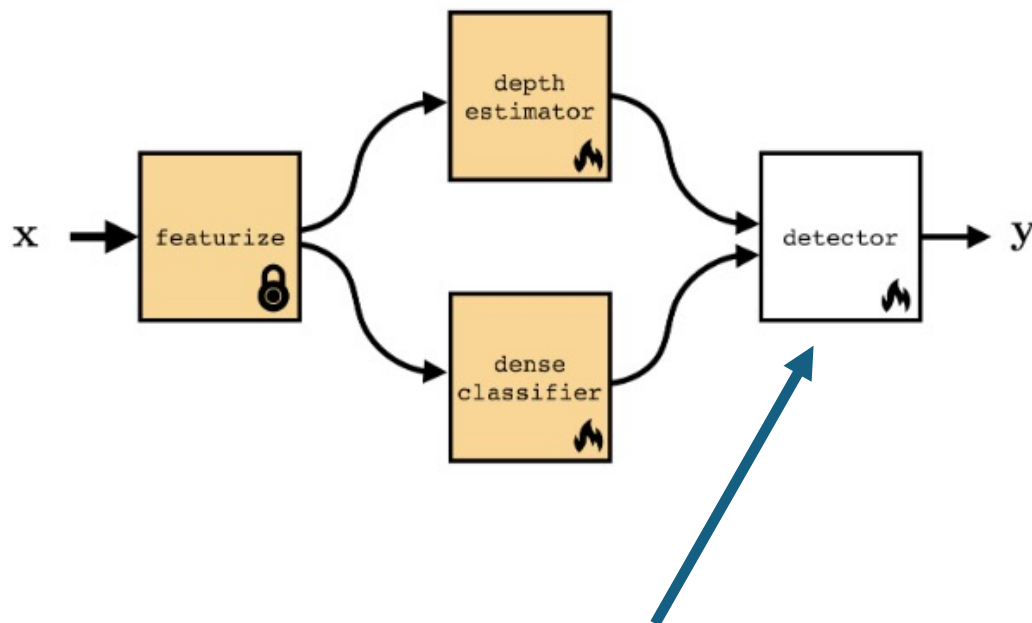


Figure 37.2

Build a larger model from multiple models



detector uses classification information and depth information.



Figure 37.3: Finetuning certain subsets of a computation graph. Here we show some modules that have been pretrained (those with a shaded background) and one that is trained from scratch at finetuning time.

<https://visionbook.mit.edu/>

Learn from another model's ground truth:

Knowledge Distillation

Supervised
learning

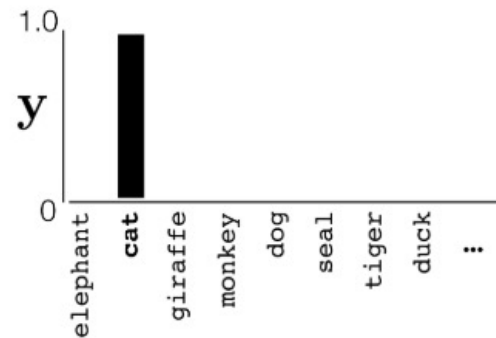
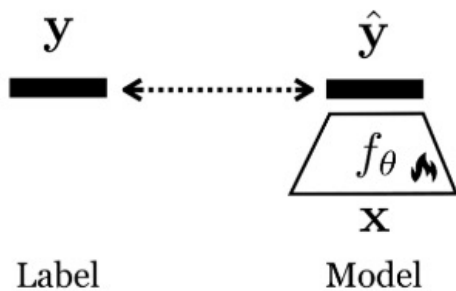
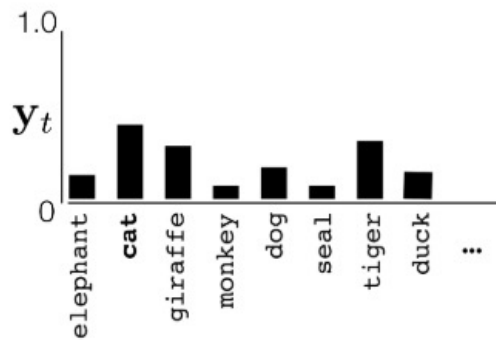
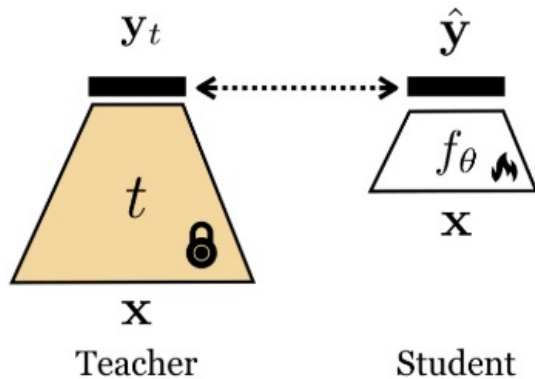


Figure 37.4: Comparing knowledge distillation to supervised learning from labels.

Knowledge
distillation



<https://visionbook.mit.edu/>

Learning Pretext Tasks: Learn Encoder $f(\cdot)$ & Decoder $g(\cdot)$

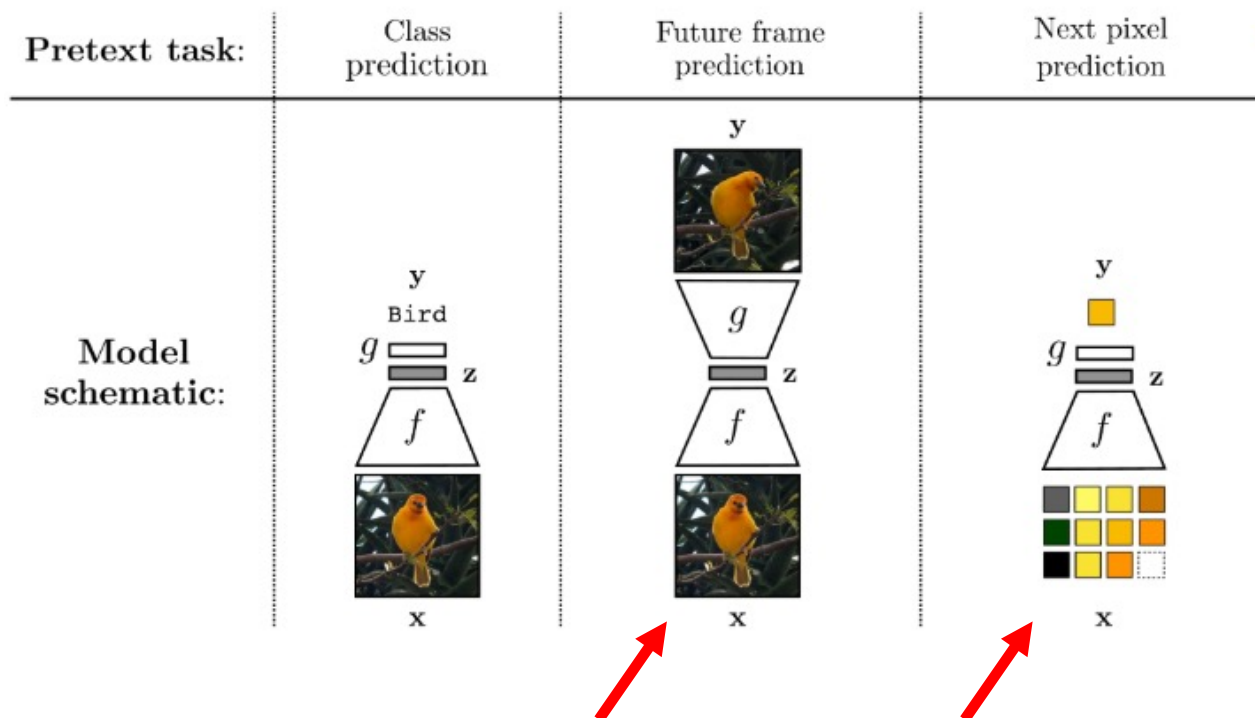


Figure 30.6: Examples of different pretext tasks.

Encoders:

$f(\cdot)$ reduces input dimensionality.

Decoders:

$g(\cdot)$ increases input dimensionality.

z : intermediate representation.

Self-supervised learning: does not require any labels!

<https://visionbook.mit.edu/>

Imputation is great for self-supervised learning!

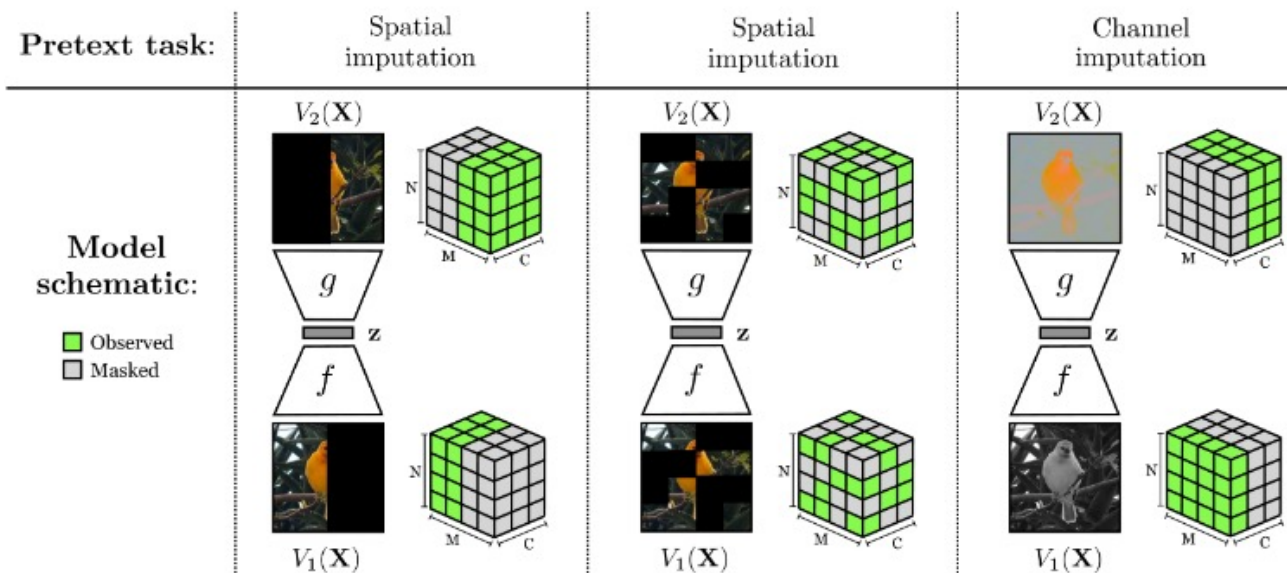


Figure 30.8: Many common pretext tasks are special cases of imputation on missing values in the data tensor.

Encoders:
 $f(\cdot)$ reduces input dimensionality.

Decoders:
 $g(\cdot)$ increases input dimensionality.

z : intermediate representation.

No labels required!

<https://visionbook.mit.edu/>

Contrastive Learning: Learning based on distinct examples (of a category)

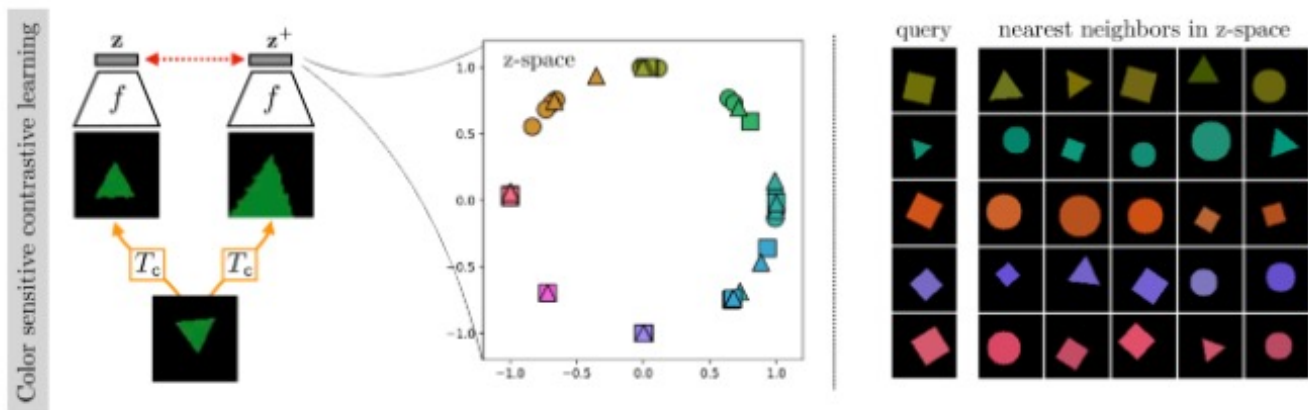
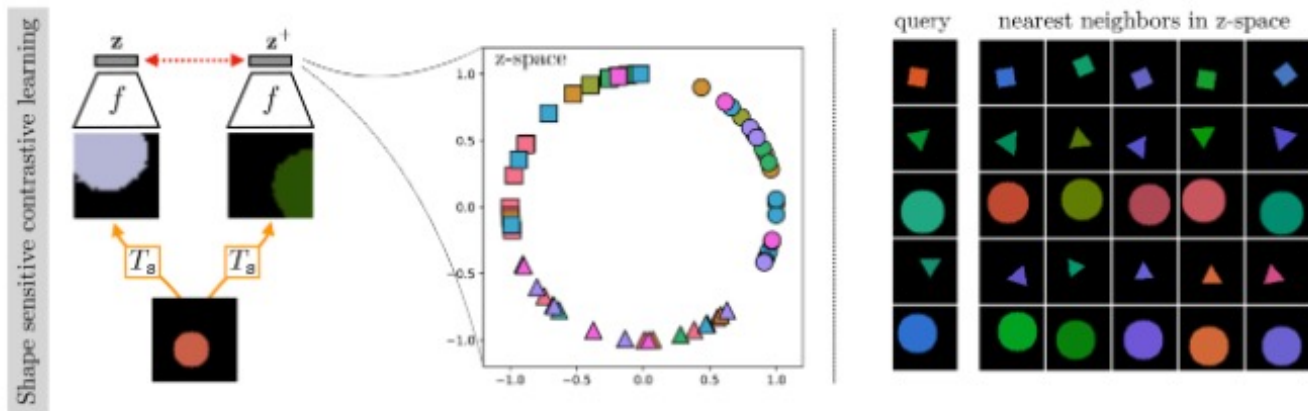


Figure 30.14: Contrastive learning on colored shapes using two different transformations for creating positive pairs. The choice of transformation controls which features the embedding becomes sensitive to and which it becomes invariant to.



<https://visionbook.mit.edu/>

Contrastive Learning: Loss functions to optimize

$$\mathcal{L}(\mathbf{x}, \mathbf{x}^+, \mathbf{x}^-) = \max(D(f(\mathbf{x}), f(\mathbf{x}^+) - m_{\text{pos}}, 0) - \max(m_{\text{neg}} - D(f(\mathbf{x}), f(\mathbf{x}^-)), 0) \quad \triangleleft \quad \text{Hadsell et al. "contrastive"}$$

$$\mathcal{L}(\mathbf{x}, \mathbf{x}^+, \mathbf{x}^-) = \max(D(f(\mathbf{x}), f(\mathbf{x}^+)) - D(f(\mathbf{x}), f(\mathbf{x}^-)) + m, 0) \quad \triangleleft \quad \text{triplet} \quad (30.4)$$

$$\mathcal{L}(\mathbf{x}, \mathbf{x}^+, \{\mathbf{x}_i^-\}_{i=1}^N) = -\log \frac{e^{f(\mathbf{x})^\top f(\mathbf{x}^+)/\tau}}{e^{f(\mathbf{x})^\top f(\mathbf{x}^+)/\tau} + \sum_i e^{f(\mathbf{x})^\top f(\mathbf{x}_i^-)/\tau}} \quad \triangleleft \quad \text{InfoNCE}$$

$\mathbf{x}$ represents a new sample.

$\mathbf{x}^+$ represents a sample of the **same category** as $\mathbf{x}$.

$\mathbf{x}^-$ represents a sample of a **different category** from $\mathbf{x}$

D is a distance function.

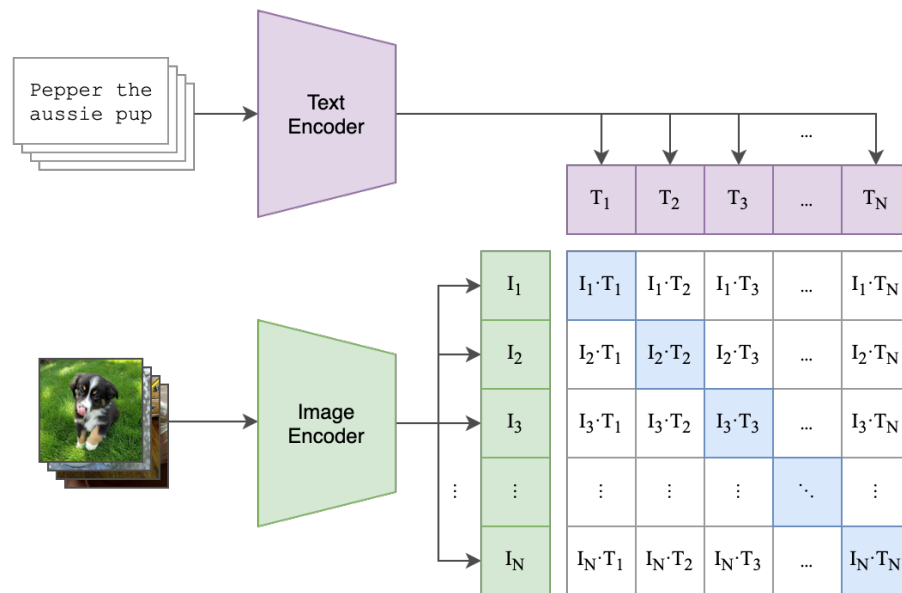
m is used for different constants.

τ is another constant.

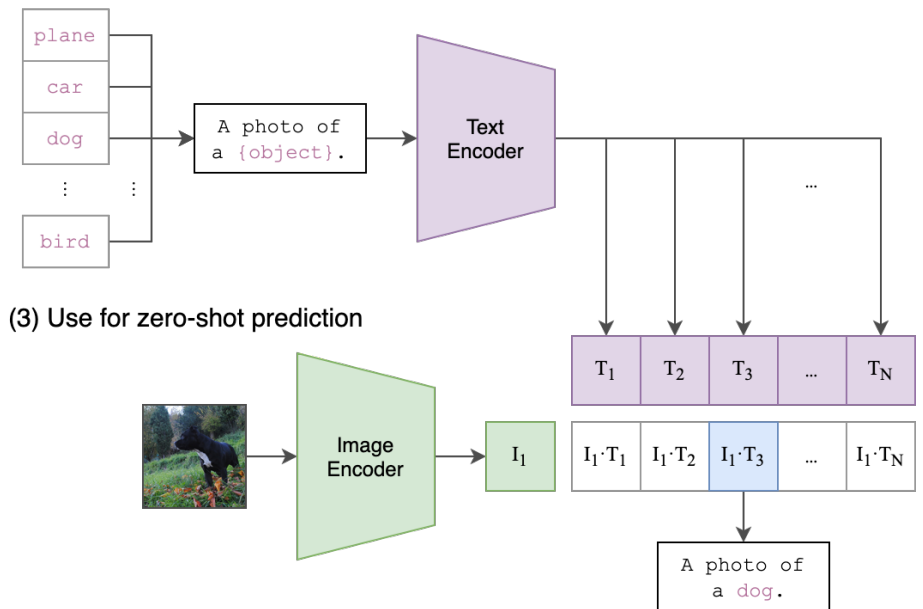
<https://visionbook.mit.edu/>

Contrastive Language-Image Pre-Training (CLIP)

(1) Contrastive pre-training



(2) Create dataset classifier from label text



(3) Use for zero-shot prediction

CLIP is instructed in natural language to predict the most relevant text snippet, given an image, without directly optimizing for the task, similarly to the zero-shot capabilities of GPT-2 and 3.

<https://github.com/openai/CLIP>

Learn a prompt!

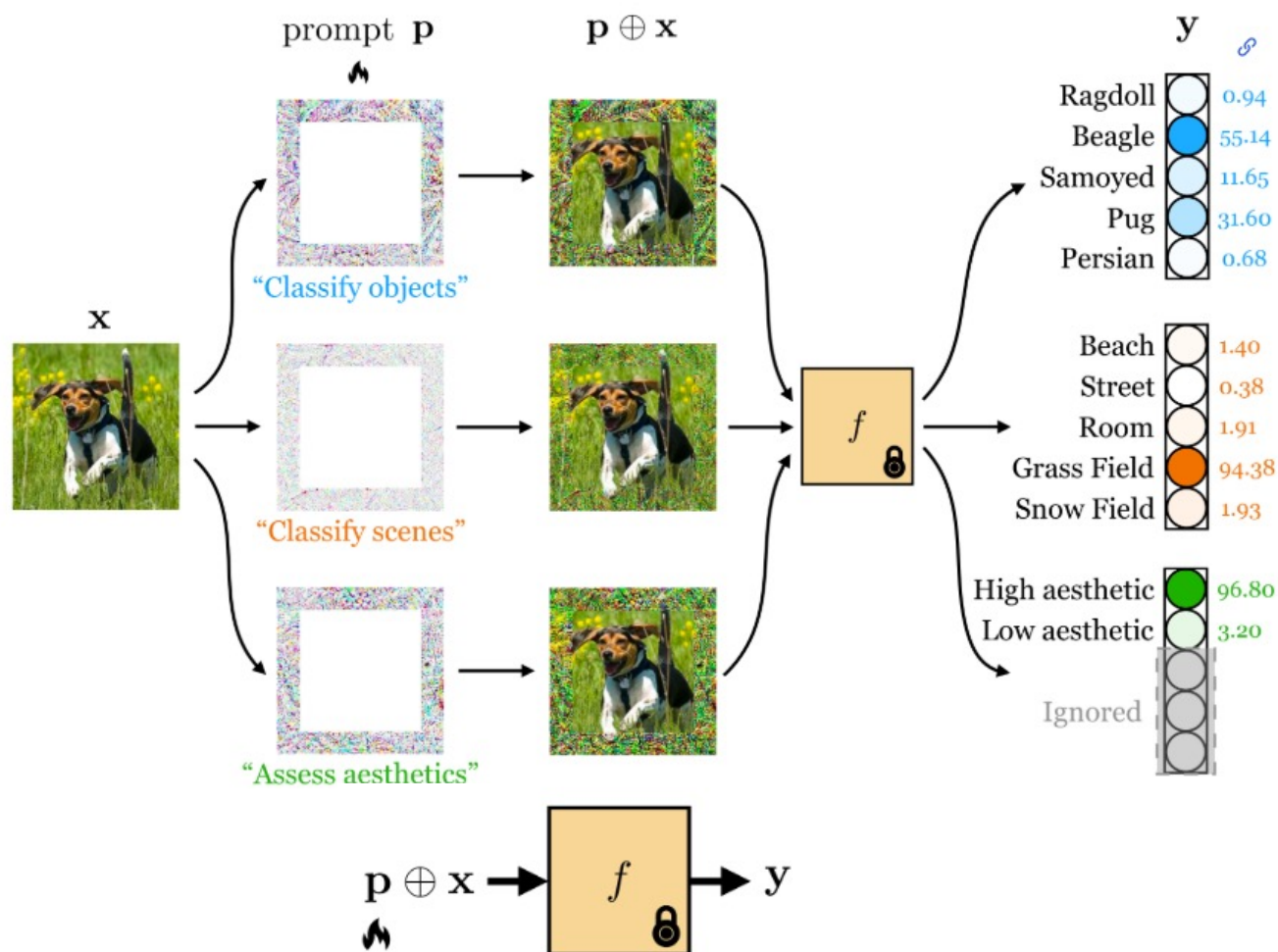


Figure 37.6: A prompt can be made out of pixels: a prompt image (which looks like a border of noise here) is added to the input image in order to change the model's behavior. Three different prompts are shown, one that adapts the model to perform scene classification, a second for aesthetics classification, and a third for object classification. Modified from [8]

$$\arg \min_{\mathbf{p}} \sum_{i=1}^M \mathcal{L}(f(\mathbf{x}^{(i)} + \mathbf{p}), \mathbf{y}^{(i)})$$

<https://visionbook.mit.edu/>

Learn an input prompt token for transformers!

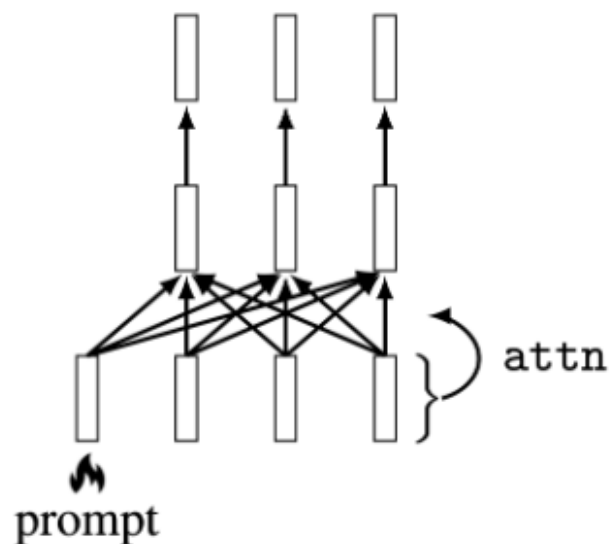


Figure 37.7: Prompting a transformer with a learnable input token. The prompt token is mixed into the network via attention; therefore parameters of the original network have to be updated (see [Chapter 26](#)).

Only applies to transformer architectures (coming up next).

<https://visionbook.mit.edu/>

Transformer Architectures

- Tokens and Token Nets
- ViT
- Attention

Generate tokens from patches

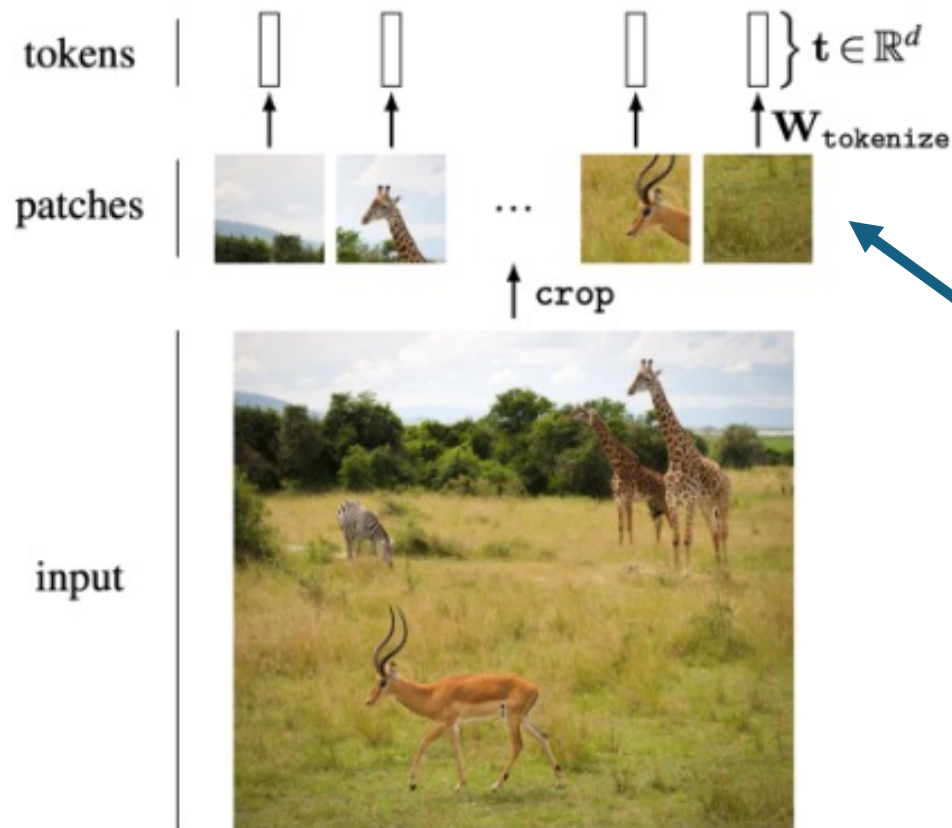


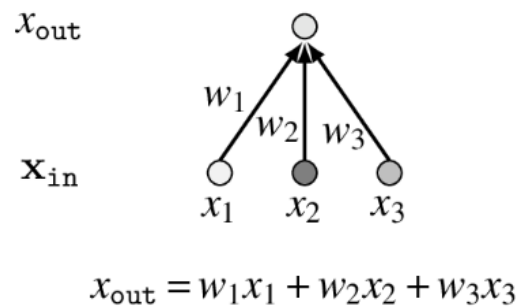
Figure 26.2: Tokenization: converting an image to a set of vectors. $\mathbf{W}_{\text{tokenize}}$ is a learnable linear projection from the dimensionality of the vectorized crops to d dimensions. This is just one of many possible ways to tokenize an image.

For videos, we use video segments!

<https://visionbook.mit.edu/>

Output Token is a Linear Combination of Input Tokens

Linear combination of neurons



Linear combination of tokens

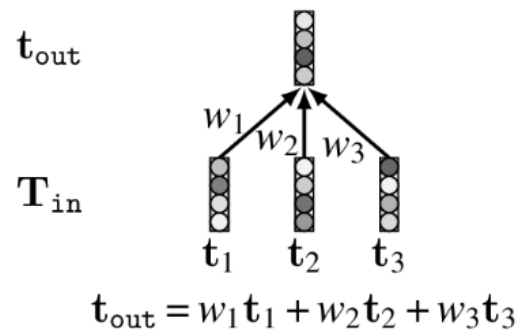


Figure 26.4: Linear combination of neurons versus tokens.

Token Net is Multi-Layered

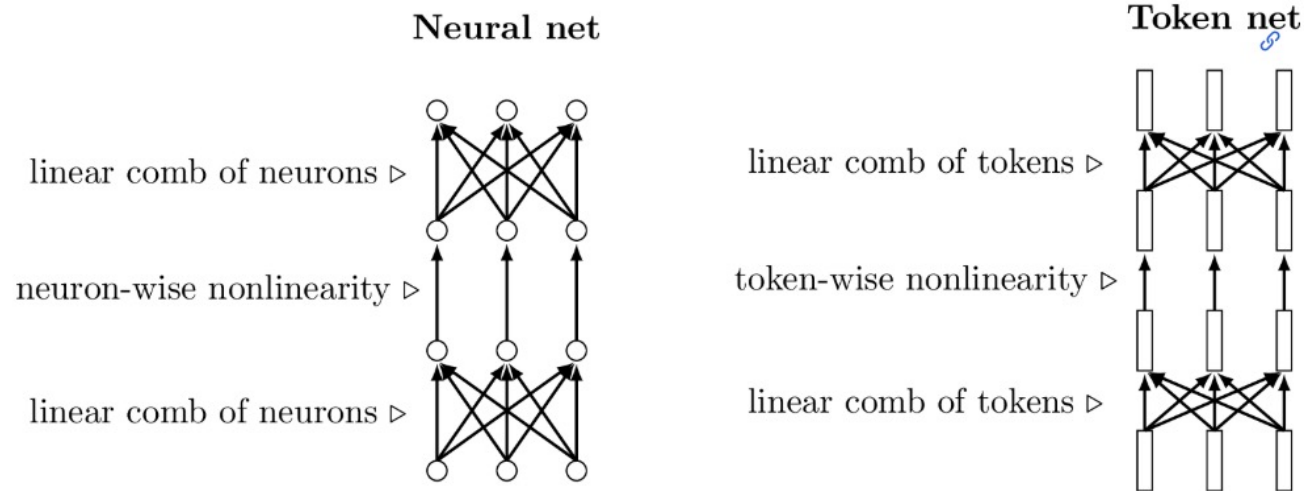


Figure 26.5: Neural nets versus token nets. The arrows here represent any functional dependency between the nodes (note that different arrows represent different types of functions).

Self Attention Layer

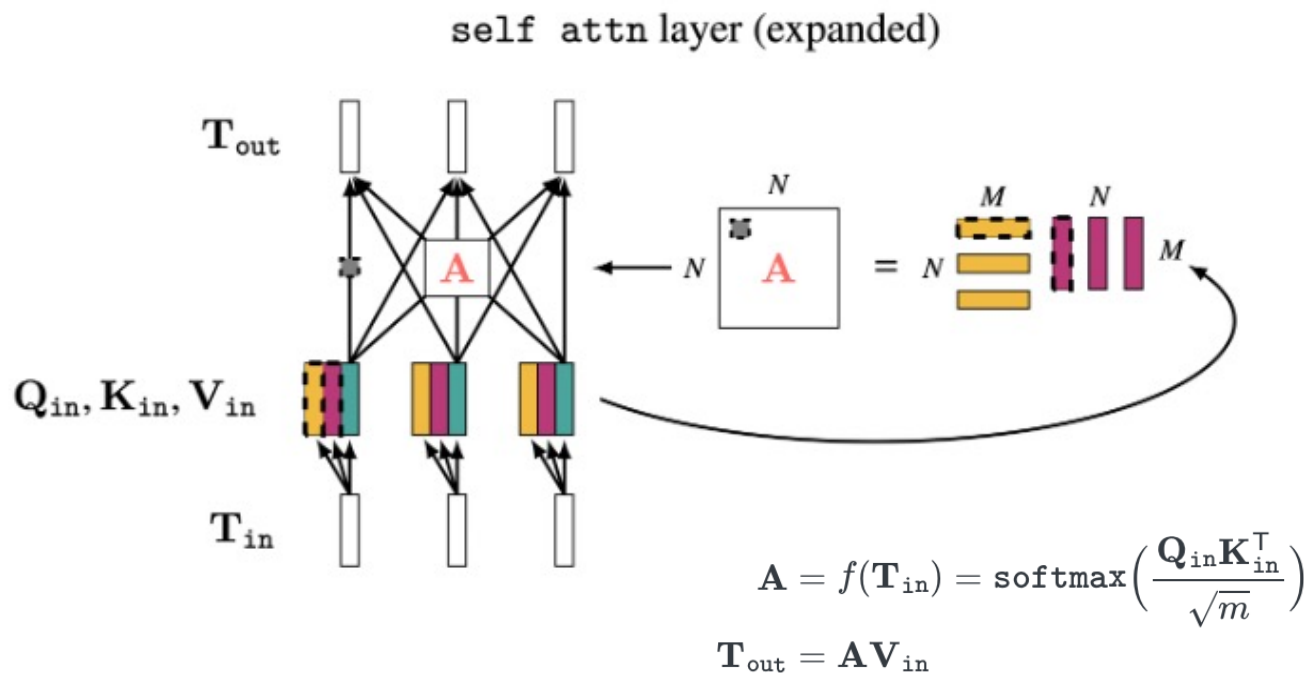


Figure 26.10: Self-attention layer expanded. The nodes with the dashed outline correspond to each other; they represent one query being matched against one key to result in a scalar similarity value, in the gray box, which acts as a weight in the weighted sum computed by $\mathbf{A}$.

◁ attention matrix (26.6)

<https://visionbook.mit.edu/>

The Vision Transformer

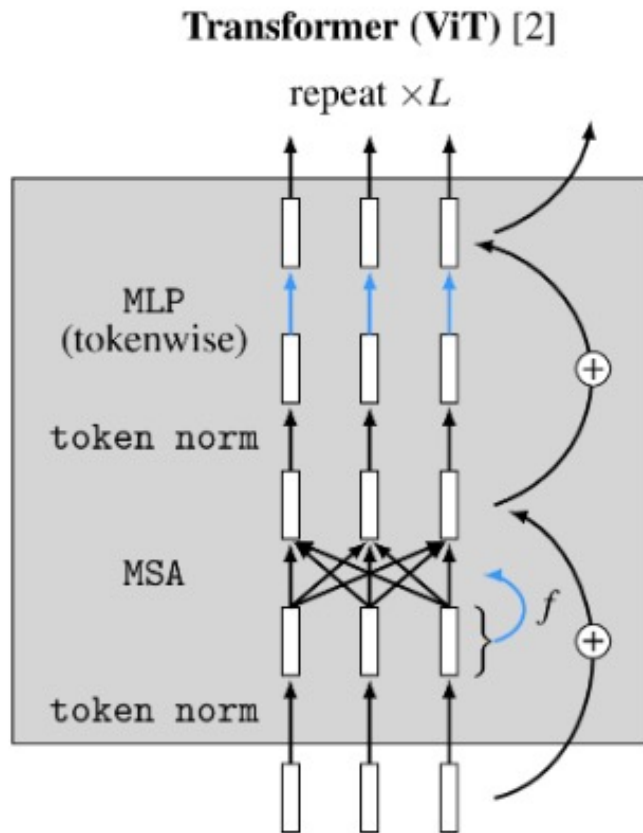


Figure 26.14: The ViT transformer architecture. This set of layers forms a computational block, shaded in gray, that can be repeated L times for a depth L ViT. To clarify where the parameters live in this architecture, we have colored all the edges with learnable parameters in blue (note that the MSA merge, f , is also learnable but not explicitly shown in this diagram).

MSA refers to multi-head Self Attention

MLP refers to multi-layer perceptron network

<https://visionbook.mit.edu/>

Adding Position Encoding

$$\begin{aligned}\mathbf{p}_x &= [\sin(x), \sin(x/B), \sin(x/B^2), \dots, \sin(x/B^P)]^\top \\ \mathbf{p}_y &= [\sin(y), \sin(y/B), \sin(y/B^2), \dots, \sin(y/B^P)]^\top \\ \mathbf{p} &= \begin{bmatrix} \mathbf{p}_x \\ \mathbf{p}_y \end{bmatrix}\end{aligned}\tag{26.12}$$

where x and y are the coordinates of the token. This representation is visualized in [Figure 26.20](#):

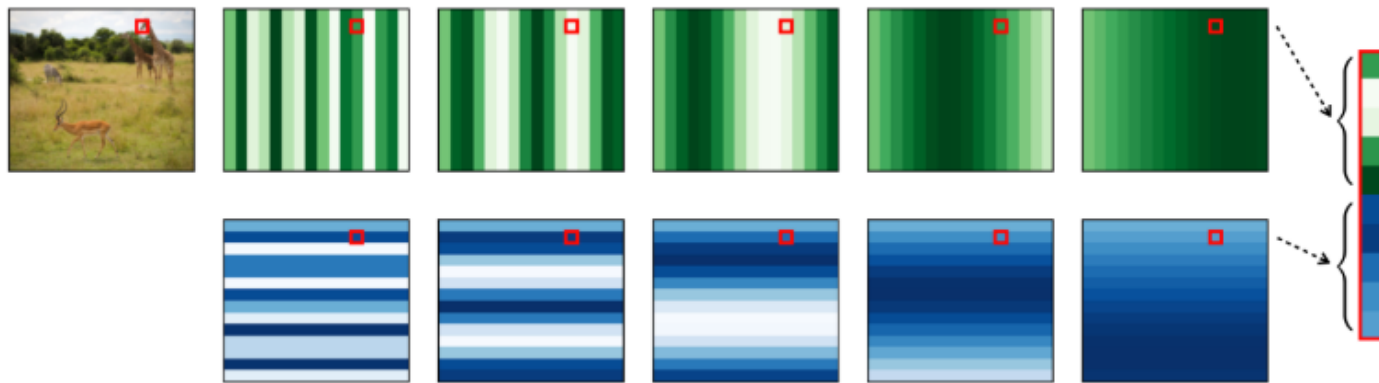


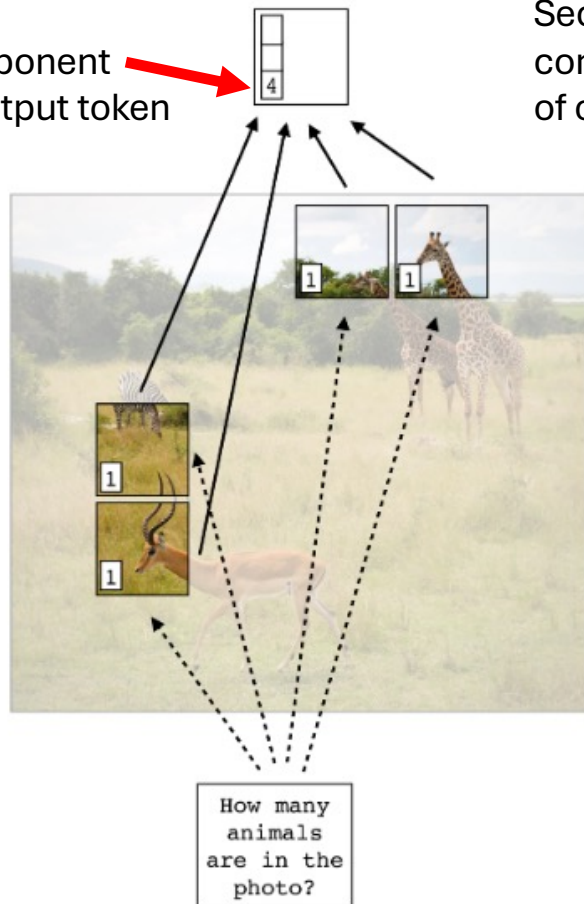
Figure 26.20: Positional codes.

Another strategy is to simply let the positional codes be *learned* by the model, which could potentially result in a better representation of space than sinusoidal codes [2].

<https://visionbook.mit.edu/>

Attention based on Query

First
component
of output token



Second
component
of output token

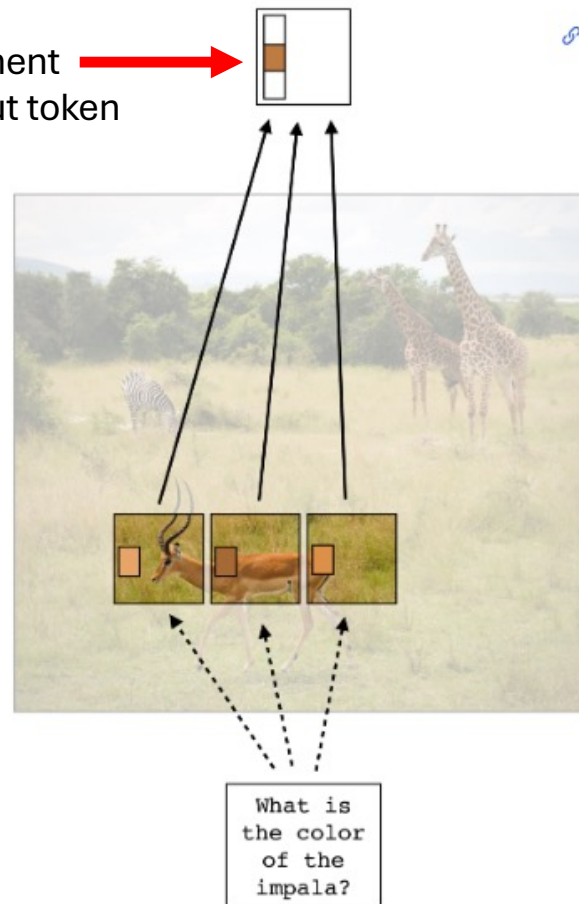
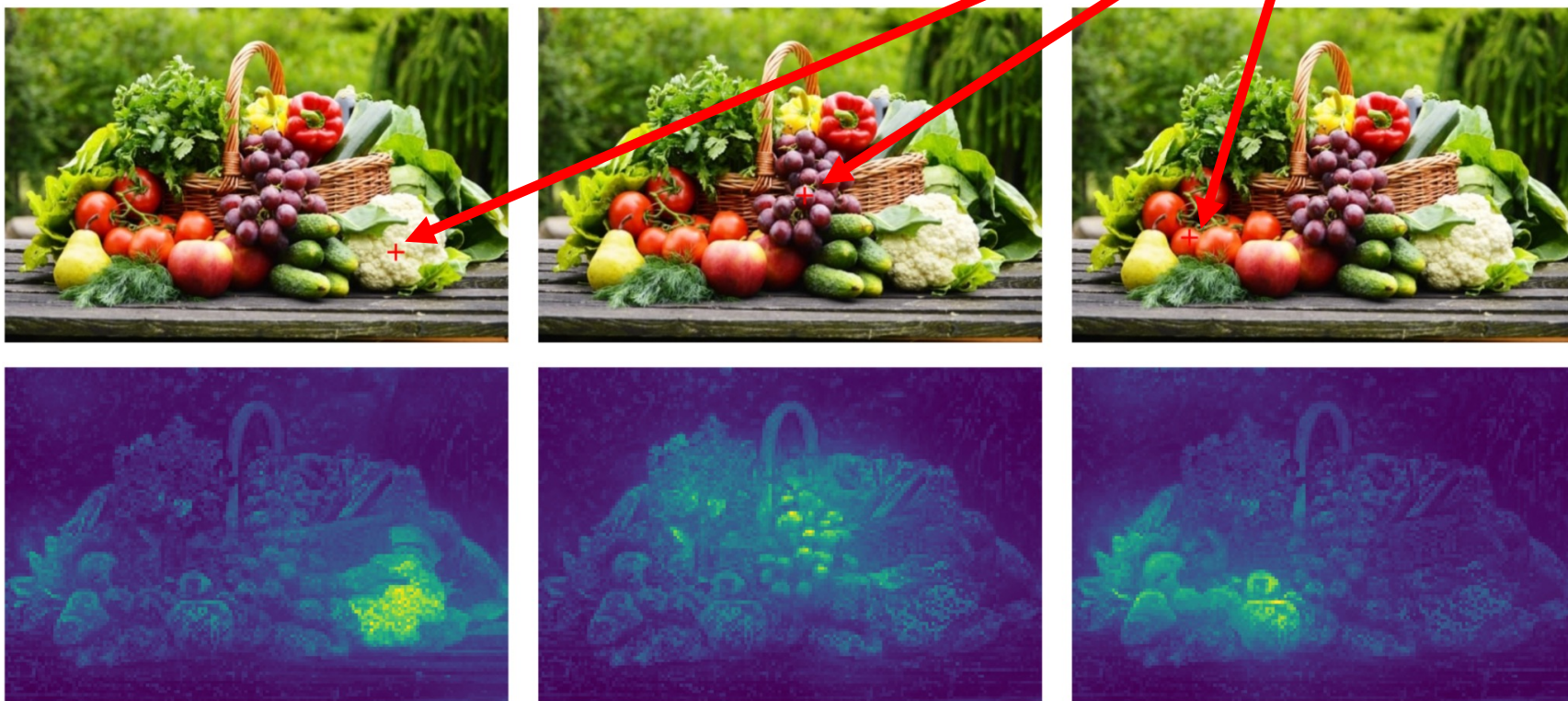


Figure 26.7: How attention can be allocated across different regions (tokens) in an image. The token code vectors consist of multiple dimensions and each can encode a different attribute of the token. To the left we show a dimension that encodes number of animal heads. To the right we show a different dimension that encodes color (or this could be three dimensions, coding RGB). The output token is a weighted sum over all the tokens attended to.

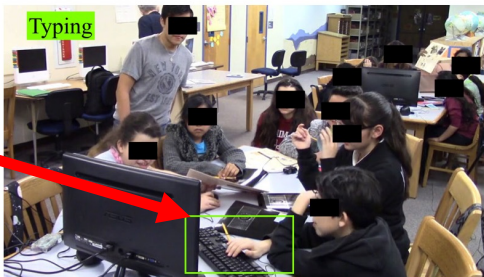
<https://visionbook.mit.edu/>

Attention Example Based on Query



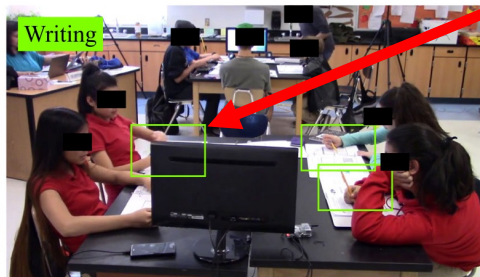
Locating Activities in Digital Video

Spatial location
by typing query
example

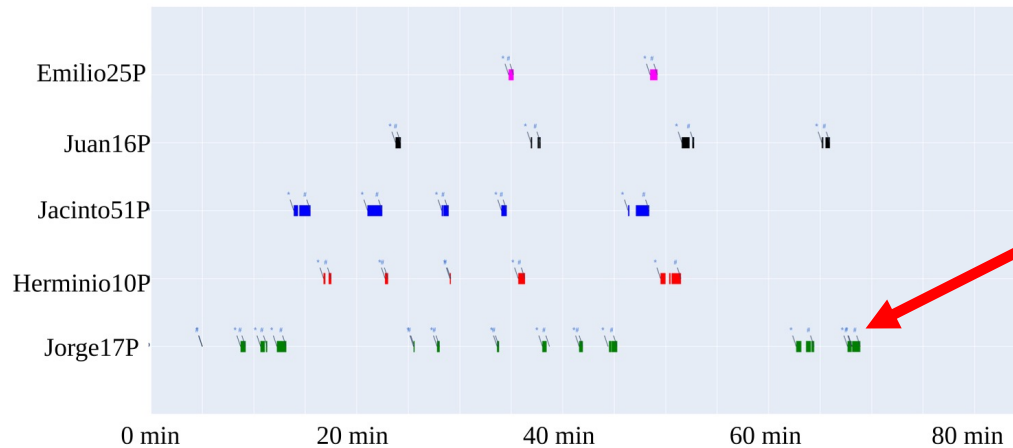


(a) Typing activity in collaborative learning environment. The keyboard is partially visible, and the students are very close to each other.

Spatial location
by writing query
example



(b) Writing activity in collaborative learning environment. Multiple writing activities with complete or partial occlusion are happening in this example.

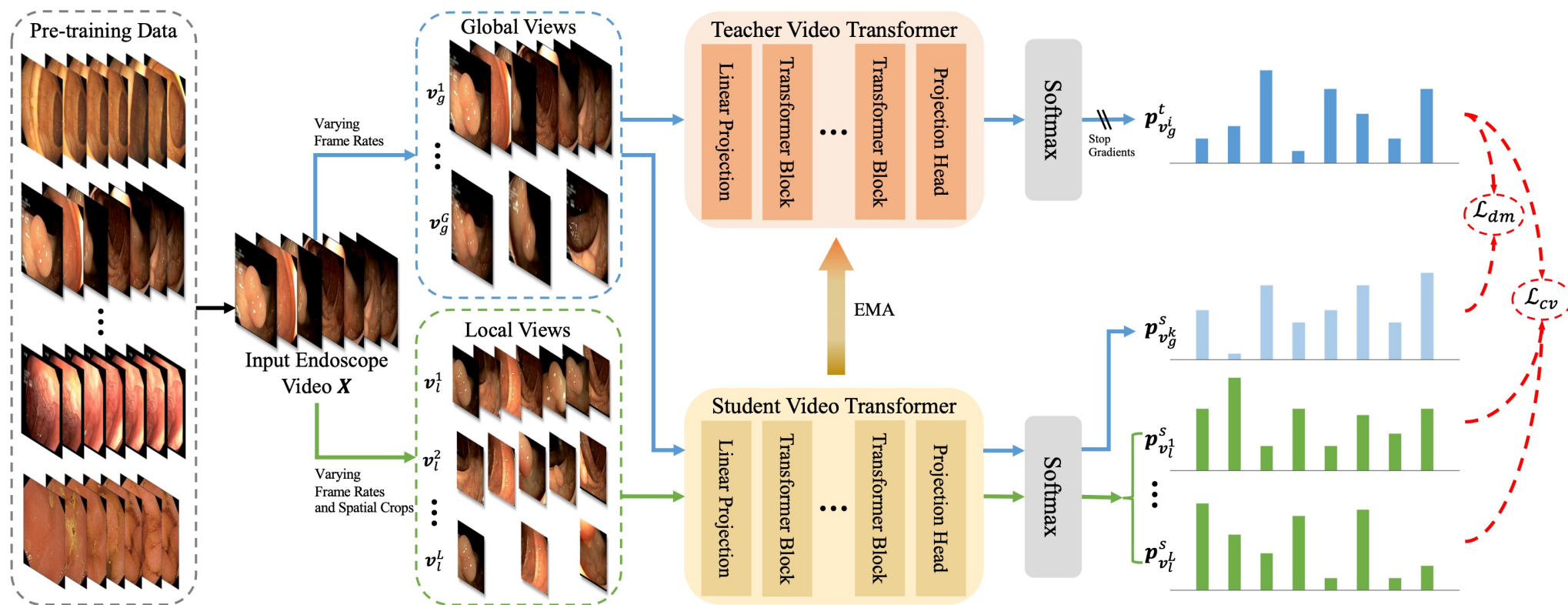


Timing location example
for typing and by person.

V. Jatla et al., "Fast and Accurate Video Analysis and Visualization of Classroom Activities Using Multiobjective Optimization of Extremely Low-Parameter Models," in *IEEE Access*, vol. 13, pp. 81143-81166, 2025, doi: 10.1109/ACCESS.2025.3567437.

A Foundation Model for Endoscopy

Foundation Model for Endoscopy Video Analysis (I of III)



<https://github.com/openmedlab/Endo-FM>

EMA is the Exponential Moving Average used to smooth the loss function.

Foundation Model for Endoscopy Video Analysis (II of III)

Table 1. Details of all pre-train and downstream datasets used in this work.

Phase	Dataset	Provider	Videos	Frames	Protocol	Disease
Pre-train	Colonoscopic [19]	CNRS	210	36534	colonoscope	adenoma, hyperplasia
	SUN [20] & SUN-SEG [13]	ANU	1018	159400	colonoscope	SSL, adenoma, hyperplasia, T1b
	LDPolypVideo [18]	USTC	237	40186	colonoscope	polyp
	Hyper-Kvasir [5]	Simula	5704	875940	gastroscope	barrett’s oesophagus, polyp, cancer
	Kvasir-Capsule [31]	Simula	1000	158892	gastroscope	erosion, erythema, etc.
	CholecTriplet [24]	BIDMC	580	90444	laparoscope	cholecystectomy
	Ours	Baoshan Branch of Renji Hospital	16494 7653	2491952 1170753	colonoscope gastroscope	polyp, erosion, etc.
Summary		6 providers	32896	5024101	3 protocols	10+ diseases
Downstream	PolypDiag [33]	Adelaide	253	485561	gastroscope	polyp, cancer
	CVC-12k [2]	UAB	29	612	colonoscope	polyp
	KUMC [15]	Kansas	53	19832	colonoscope	adenoma, hyperplasia
	Summary	3 providers	335	506005	2 protocols	4 diseases

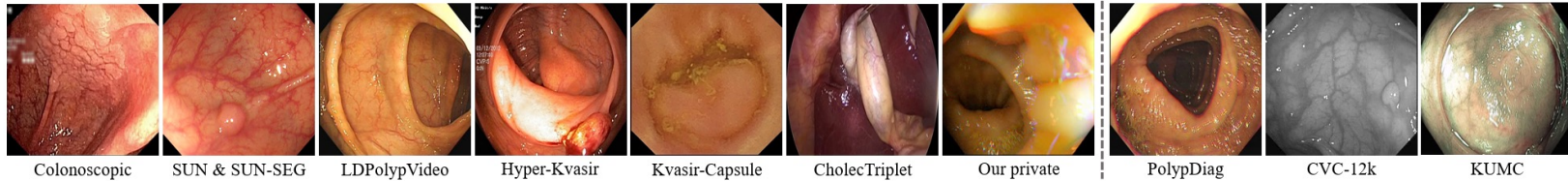


Fig. 2. Example frames of the 10 pre-train and downstream datasets used in this work.

Foundation Model for Endoscopy Video Analysis (III of III)

Table 2. Comparison with other latest SOTA methods on 3 downstream tasks. We report F1 score (%) for PolypDiag, Dice (%) for CVC-12k, and F1 score (%) for KUMC.

Method	Venue	Pre-training Time (h)	PolypDiag (Classification)	CVC-12k (Segmentation)	KUMC (Detection)
Scratch (Rand. init.)		N/A	83.5 $\pm$ 1.3	53.2 $\pm$ 3.2	73.5 $\pm$ 4.3
TimeSformer [3]	ICML'21	104.0	84.2 $\pm$ 0.8	56.3 $\pm$ 1.5	75.8 $\pm$ 2.1
CORP [12]	ICCV'21	65.4	87.1 $\pm$ 0.6	68.4 $\pm$ 1.1	78.2 $\pm$ 1.4
FAME [7]	CVPR'22	48.9	85.4 $\pm$ 0.8	67.2 $\pm$ 1.3	76.9 $\pm$ 1.2
ProViCo [26]	CVPR'22	71.2	86.9 $\pm$ 0.5	69.0 $\pm$ 1.5	78.6 $\pm$ 1.7
VCL [27]	ECCV'22	74.9	87.6 $\pm$ 0.6	69.1 $\pm$ 1.2	78.1 $\pm$ 1.9
ST-Adapter [25]	NeurIPS'22	8.1	84.8 $\pm$ 0.7	64.3 $\pm$ 1.9	74.9 $\pm$ 2.9
Endo-FM (Ours)		20.4	90.7$\pm$0.4	73.9$\pm$1.2	84.1$\pm$1.3

<https://github.com/openmedlab/Endo-FM>

Generative models based on diffusion

- Forward and backward diffusion
- Data augmentation using a generative model

Learning a Data Generator

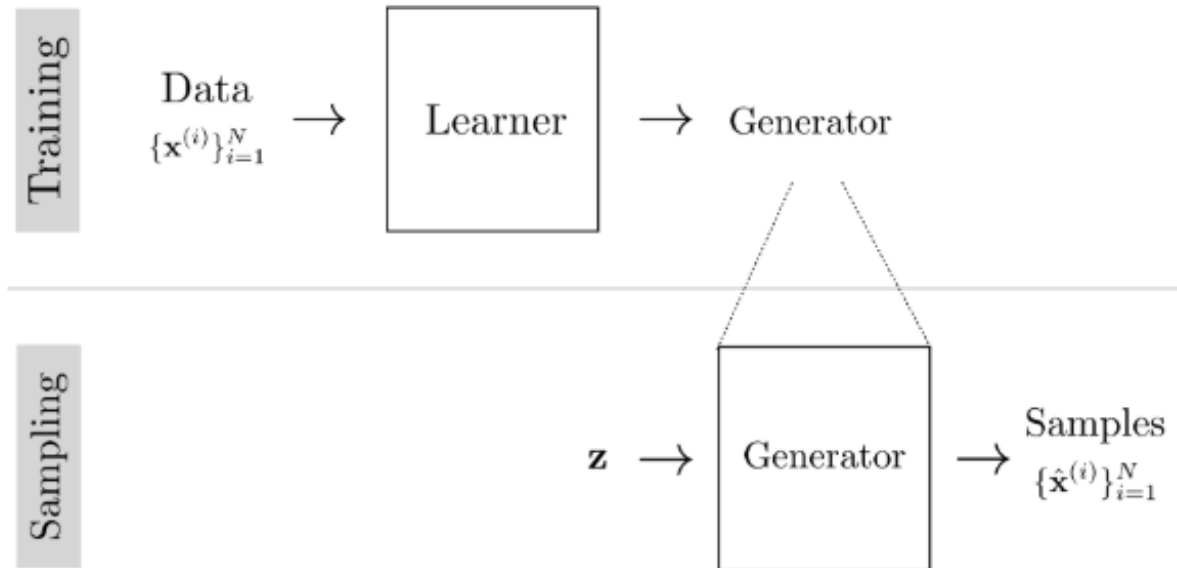


Figure 32.6: Learning and using a generator.

$\mathbf{z}$ is a representation (could be random noise).

Training dataset: Create a sequence from any input image

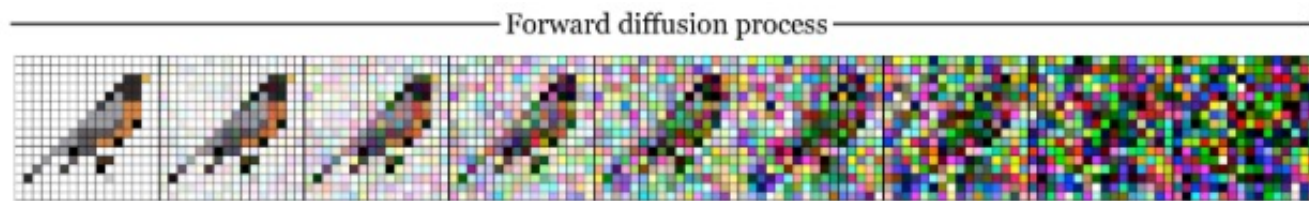


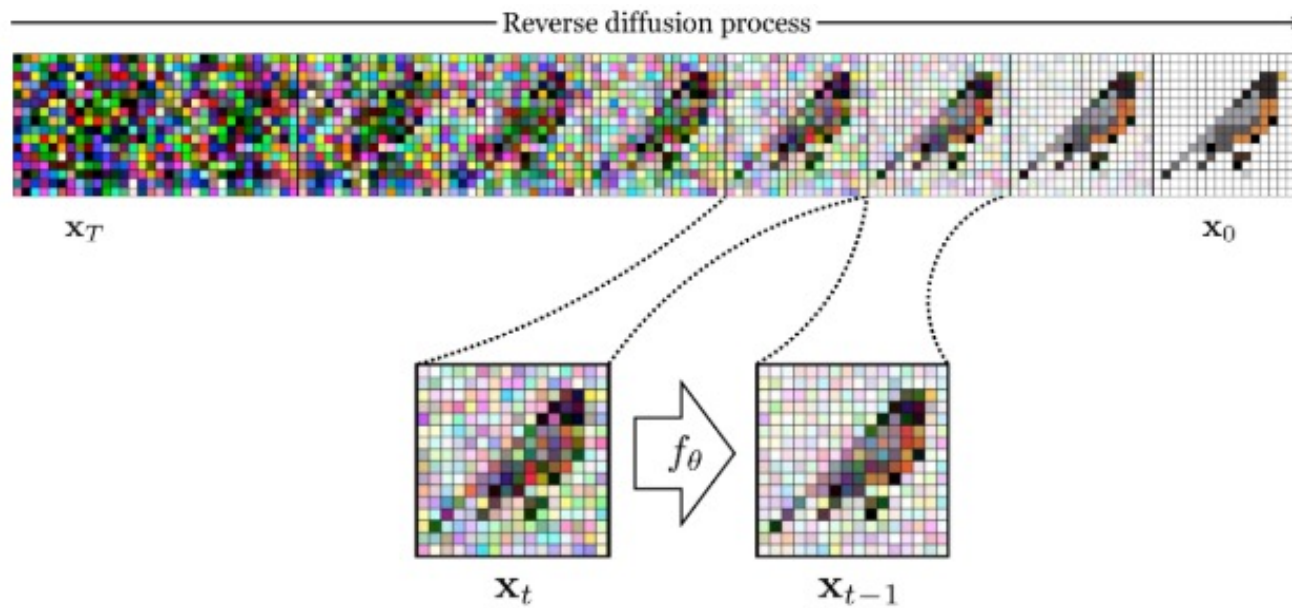
Figure 32.14: Forward diffusion process.

After T steps of this process, the image $\mathbf{x}_T$ looks like pure noise, if T is large enough. In fact, if we use the following noise process, then $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ as $T \rightarrow \infty$ (which follows from equation 4 in [4]).

$$\begin{aligned}\mathbf{x}_t &= \sqrt{(1 - \beta_t)}\mathbf{x}_{t-1} + \sqrt{\beta_t}\epsilon_t \\ \epsilon_t &\sim \mathcal{N}(\mathbf{0}, \mathbf{I})\end{aligned}$$

<https://visionbook.mit.edu/>

Neural Net Goal: Reverse the sequence



$$\hat{\mathbf{x}}_{t-1} = f_\theta(\mathbf{x}_t, t)$$

Figure 32.15: Reverse diffusion process.
The forward process creates supervision to train the reverse process.

Generative Model Based on Diffusion

Algorithm 32.2: Training a diffusion model consists of first producing training pairs of the form {noisy image, less noisy image}. Then do supervised learning on these pairs.

```
1 Input: training data  $\{\mathbf{x}^{(i)}\}_{i=1}^N$ 
2 Output: trained model  $f_\theta$ 
3 Generate training sequences via diffusion:
4 for  $i = 1, \dots, N$  do
5   for  $t = 1, \dots, T$  do
6      $\epsilon_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
7      $\mathbf{x}_t^{(i)} \leftarrow \sqrt{(1 - \beta_t)}\mathbf{x}_{t-1}^{(i)} + \sqrt{\beta_t}\epsilon_t$ 
8
9 Train denoiser  $f_\theta$  to reverse these sequences:
10  $\theta^* = \arg \min_\theta \sum_{i=1}^N \sum_{t=1}^T \mathcal{L}(f_\theta(\mathbf{x}_t^{(i)}, t), \mathbf{x}_{t-1}^{(i)})$ 
11 Return:  $f_{\theta^*}$ 
```

Algorithm 32.2: Training a diffusion model consists of first producing training pairs of the form {noisy image, less noisy image}. Then do supervised learning on these pairs.

<https://visionbook.mit.edu/>

Use an Auto-encoder as a Diffusion Model

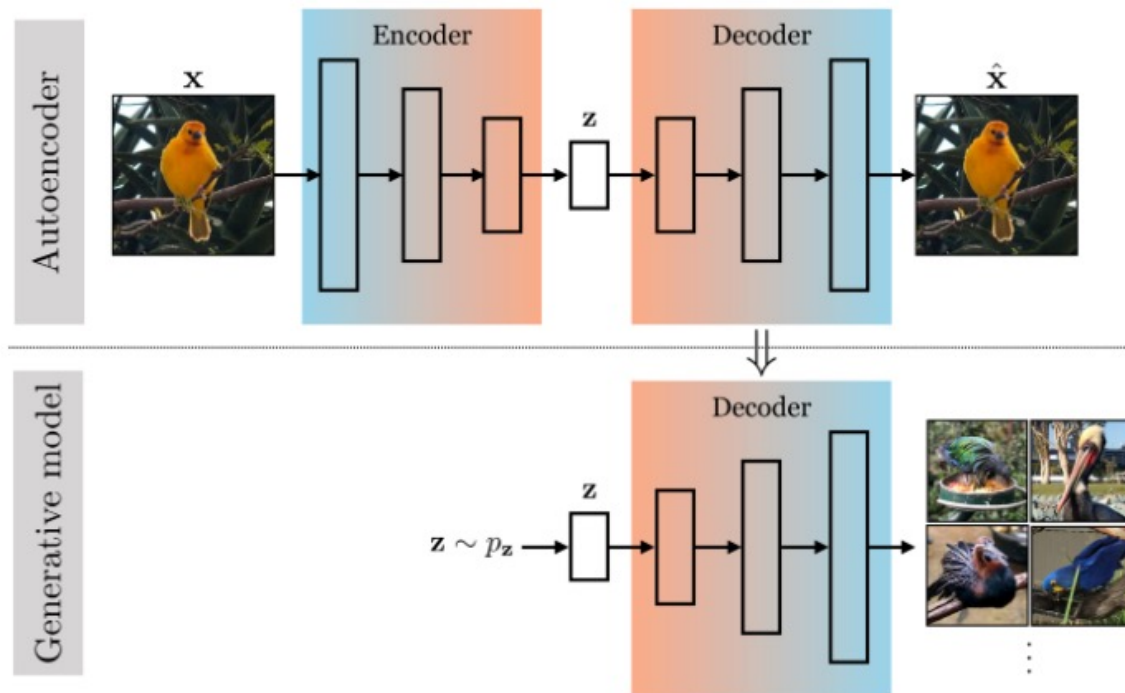


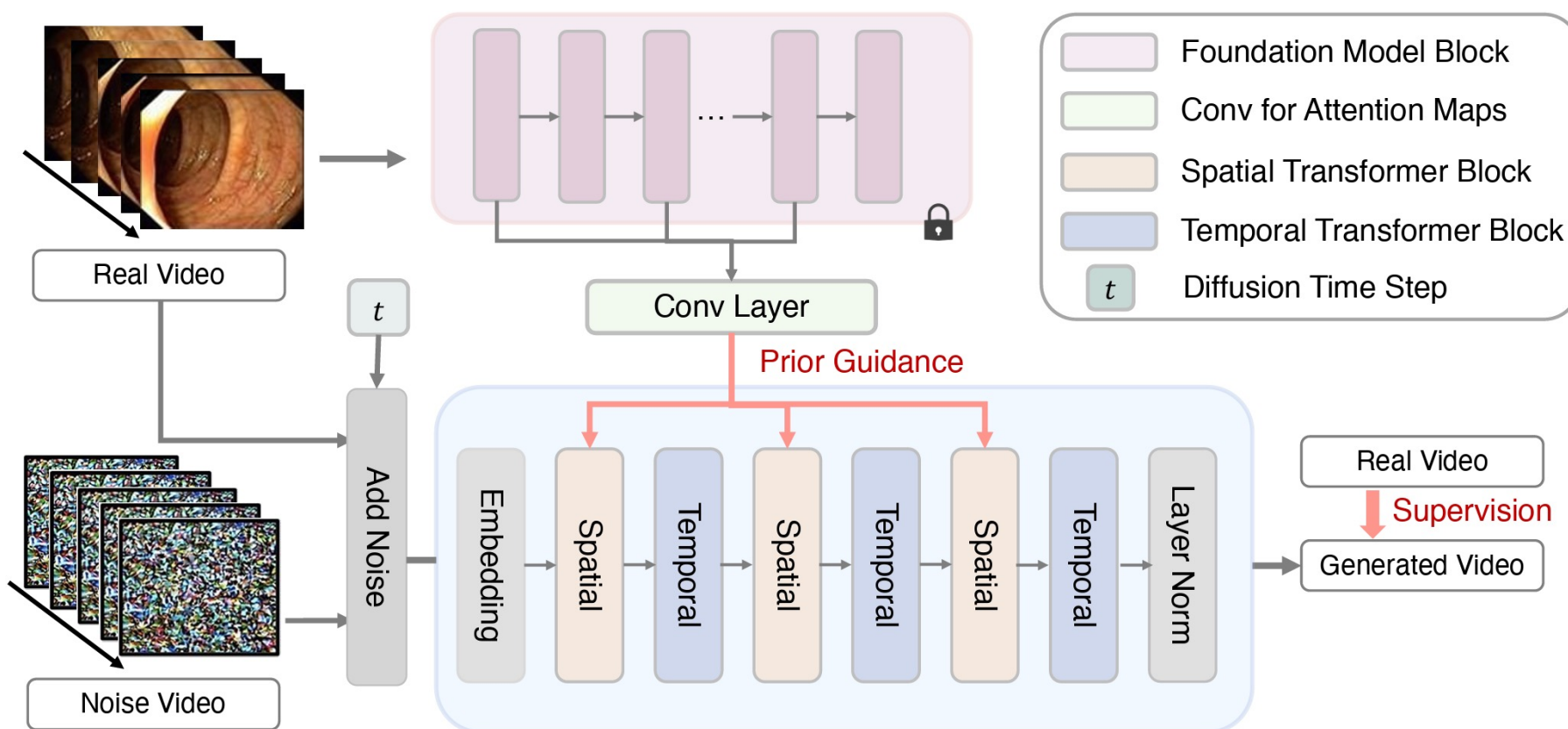
Figure 33.3: The relationship between an autoencoder and a generative model.

Data augmentation using a Generative Model

Algorithm 36.1: Generative data augmentation (genaug).

```
1 Input: Generative model  $g_\theta : \mathcal{Z} \rightarrow \mathcal{X}$  and its (approximate) inverse  $f_\psi : \mathcal{X} \rightarrow \mathcal{Z}$ , perturbation scale  
   parameter  $\sigma$ , Dataset  $X \triangleq \{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\}_{i=1}^N$ , hyperparameter  $k$   
2 Output: Augmented dataset  $\tilde{X} \triangleq \{\tilde{\mathbf{x}}^{(i)}, \tilde{\mathbf{y}}^{(i)}\}_{i=1}^{kN}$   
3 for  $i = 1, \dots, N$  do  
4   for  $j = 1, \dots, k$  do  
5      $\mathbf{z}^{(i)} = f_\psi(\mathbf{x}^{(i)})$   
6      $\mathbf{w}_j \sim \mathcal{N}(0, \sigma^2)$   
7      $\tilde{X} \leftarrow \{g_\theta(\mathbf{z}^{(i)} + \mathbf{w}_j), \mathbf{y}^{(i)}\}$ 
```

Video Generation Models for Endoscopy



“Endora: Video Generation Models as Endoscopy Simulators”. *International conference on medical image computing and computer-assisted intervention*. Cham: Springer Nature Switzerland, 2024.

Generative Adversarial Network (GAN) and Domain Adaptation

- Decoder to generate synthetic examples
- Detector of real versus synthetic
- Zero-shot learning
- Domain Adaptation

Start with a decoder to generate synthetic examples

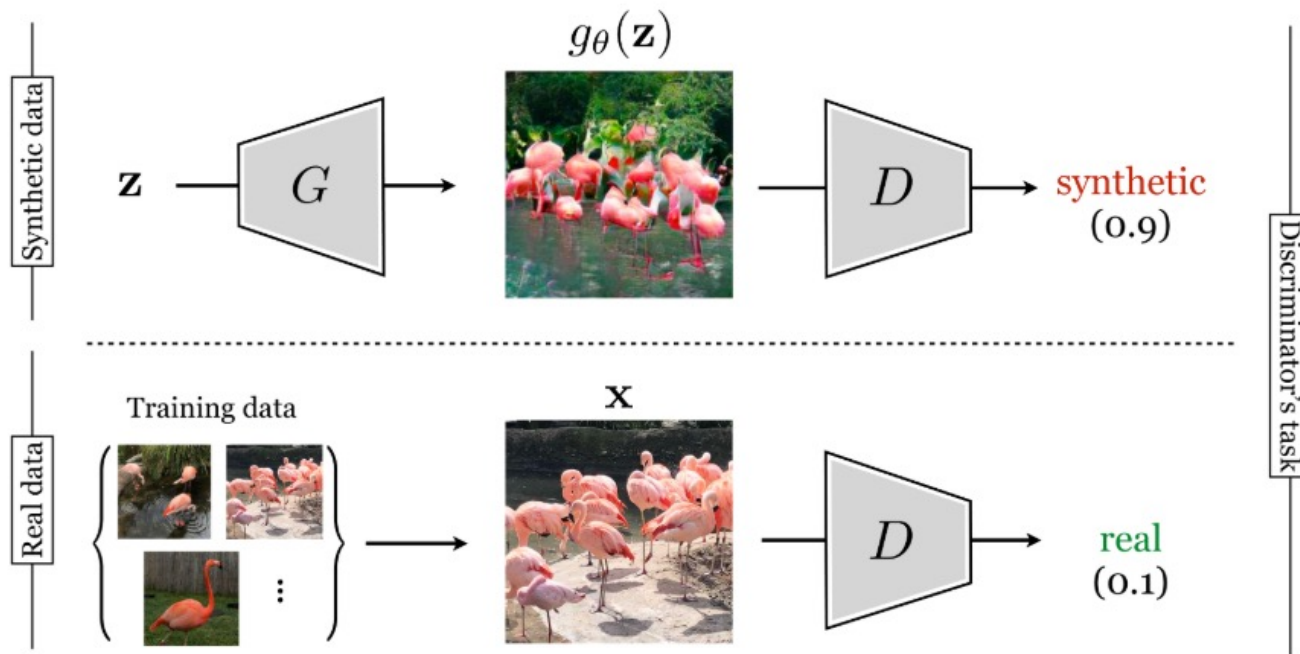


Figure 32.16: Architecture of a GAN being trained to generate images of flamingos. The synthetic image in this example is generated by BigGAN [7]

Train a detector of real versus synthetic

GANs consist of two neural networks, the **generator**, $g_\theta : \mathcal{Z} \rightarrow \mathcal{X}$, which synthesizes data from noise, and a **discriminator**, $d_\phi : \mathcal{X} \rightarrow \Delta^1$, which tries to classify between synthesized data and real data.

The g_θ and d_ϕ play an adversarial game in which g_θ tries to become better and better at generating synthetic images while d_ϕ tries to become better and better at detecting any errors g_θ is making. The learning problem can be written as a minimax game:

$$\arg \min_{\theta} \max_{\phi} \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log d_\phi(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z} [\log(1 - d_\phi(g_\theta(\mathbf{z})))] \quad (32.6)$$

<https://visionbook.mit.edu/>

Zero-shot Learning for Medical Image Classification

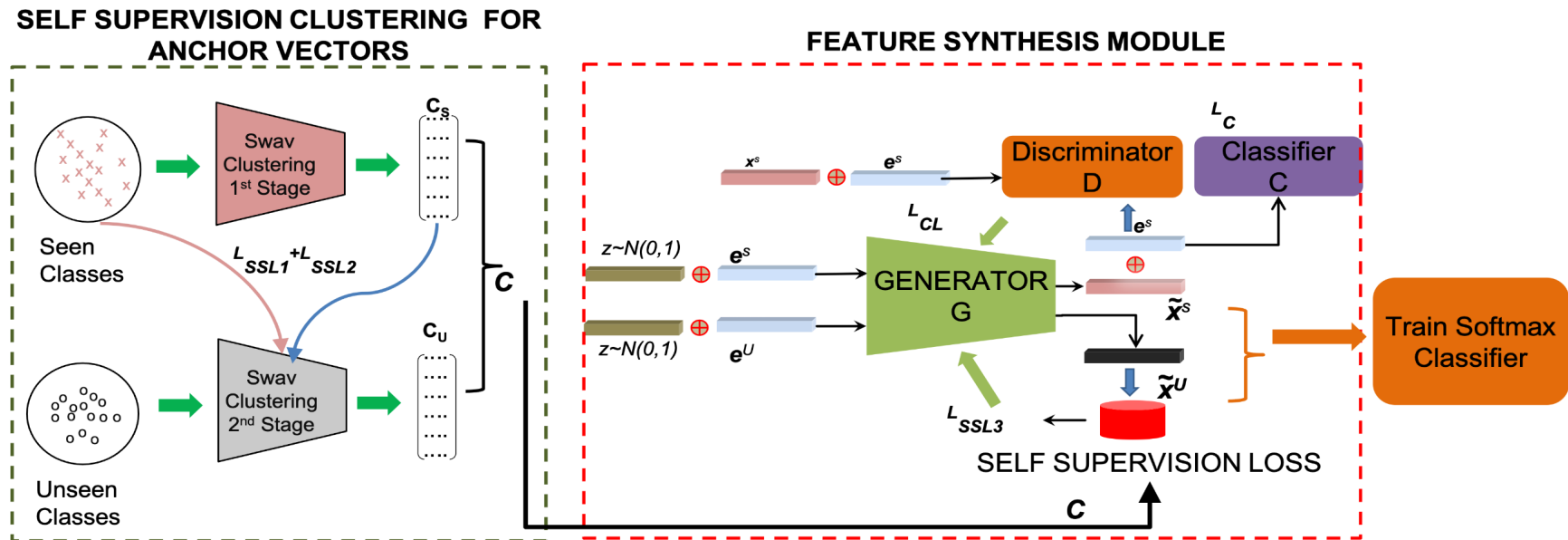


Figure 1. Architecture of proposed SC-GZSL method. In the first step we generate anchor vectors (cluster centroids) by using SSL within the SwAV clustering approach [10]. We have two clustering stages: one for Seen class samples and second for Unseen classes. Feature generation leverages one Generator and one Discriminator alongwith anchor vectors (from clustering) to derive SSL loss terms.

“Medical Image Classification Using Generalized Zero Shot Learning”, ICCV, 2021.

Domain Adaptation Algorithm

-
- 1 **Input:** training data $\{x_{\text{train}}^{(i)}, y_{\text{train}}^{(i)}\}_{i=1}^N$, test data, $\{x_{\text{test}}^{(i)}\}_{i=1}^M$
 - 2 **Output:** trained model F
 - 3 **Train predictor on train:** $f = \arg \min_f \mathbb{E}_{x_{\text{train}}, y_{\text{train}}} [\mathcal{L}(f(x_{\text{train}}), y_{\text{train}})]$
 - 4 **Train translator $\mathcal{X}_{\text{test}}$ to $\mathcal{X}_{\text{train}}$:**
 $G = \arg \min_g \max_d \mathbb{E}_{x_{\text{train}}} [\log d(x_{\text{train}})] + \mathbb{E}_{x_{\text{test}}} [\log(1 - d(g(x_{\text{test}})))]$
 - 5 return $F = f \circ g$
-

Algorithm 37.2: Unpaired domain adaptation via adversarial translation.

Can also do the opposite: translate training data to test data.

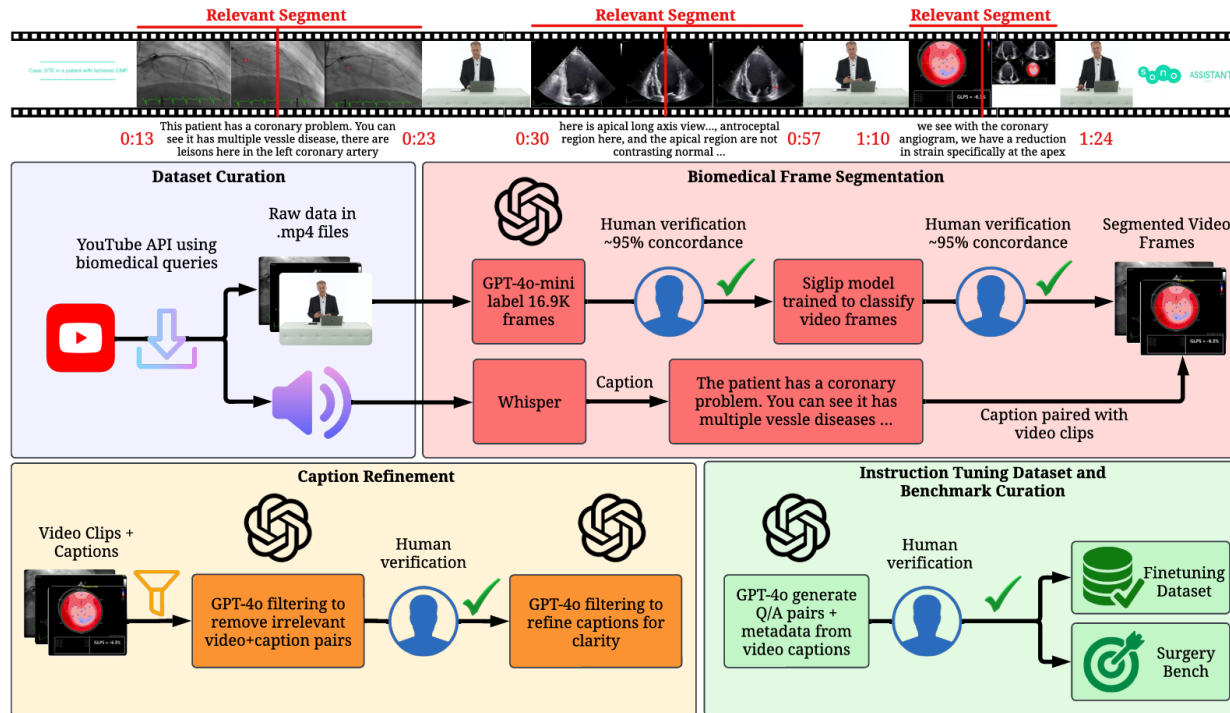
Example: **Realistic simulation can adapt the training data to test data.**

<https://visionbook.mit.edu/>

An Instructional Medical Video Example

- Dataset curation
- Dataset distribution
- VQA Examples
- Vision language Models
- Results

Instructional Medical Videos: Dataset Generation



Reference:

Thapa, Rahul, et al. "How Well Can General Vision-Language Models Learn Medicine By Watching Public Educational Videos?." *arXiv preprint arXiv:2504.14391* (2025).

OpenBiomedVid: 1031 hours of video-caption and Q/A pairs, curated through a multi-step human-in-the-loop pipeline.

MIMIC-IV-ECHO:

- 500,000 echocardiograms
- 7,243 studies
- 4,579 patients.

Figure 1: Data generation pipeline. (1) **Dataset Curation:** Videos are collected from YouTube using manually and clinically guided search queries. (2) **Biomedical Frame Segmentation:** A SigLIP model is fine-tuned using GPT-labeled data to detect biomedical frames. (3) **Caption Refinement:** Transcriptions generated using Whisper are filtered and cleaned using GPT-4o to ensure visual grounding. (4) **Instruction Data Generation:** GPT-4o is used to generate multi-turn Q/A pairs and extract structured metadata from the cleaned captions. Human verification is incorporated throughout the pipeline to ensure high data quality.

Instructional Medical Videos: Dataset Statistics

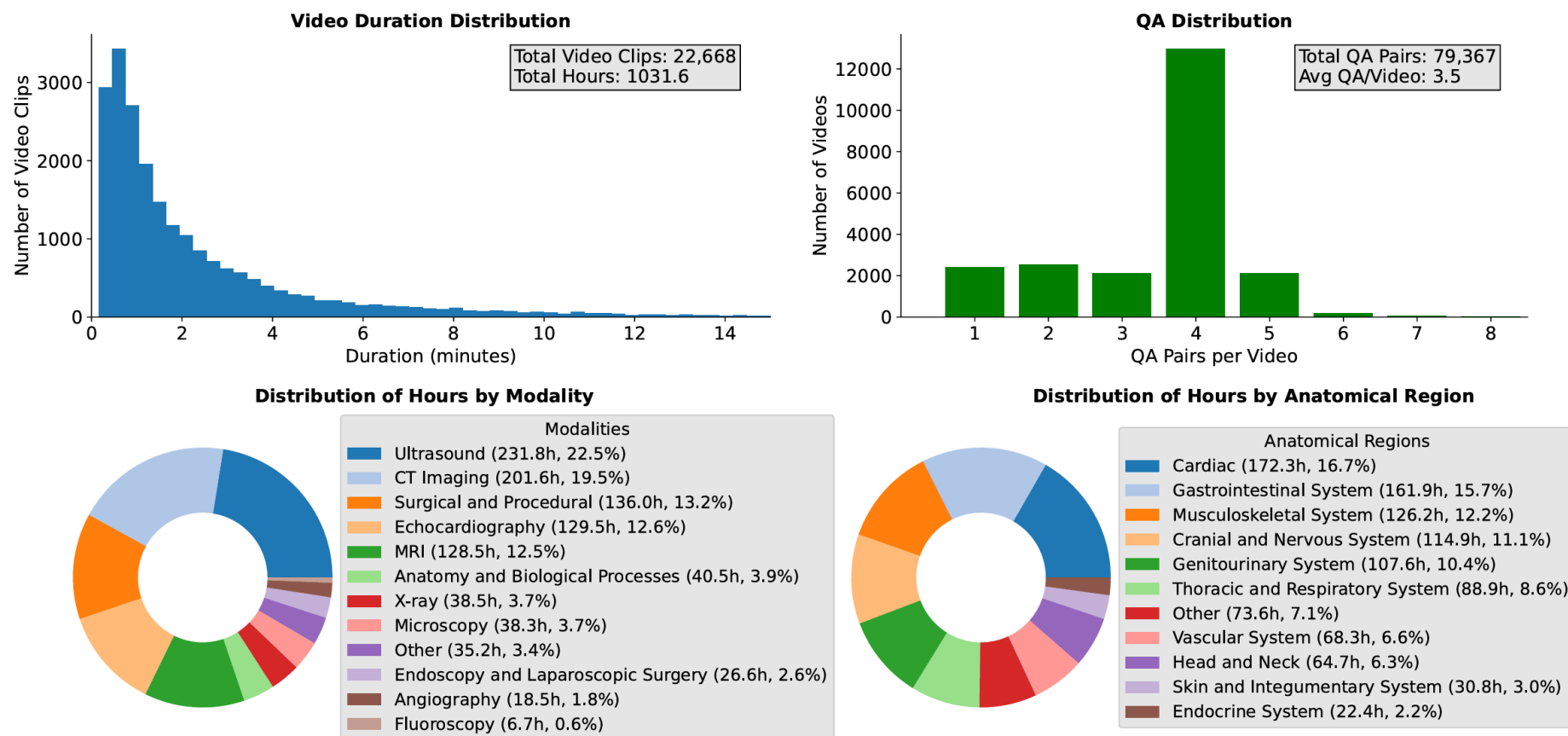


Figure 2: Distribution of the fine-tuning dataset across different biomedical modalities and anatomical regions.

Instructional Medical Videos: GPT-5 models

Flagship models

Our frontier models designed to spend more time thinking before producing a response, making them ideal for complex, multi-step problems.

GPT-5

The best model for coding and agentic tasks across industries

- ✓ Text & vision
- ✓ Reasoning capabilities
- ✓ Support for all built-in tools
- ✓ 400k context length | 128k max output tokens
- ✓ Input: \$1.25 | Output: \$10.00 per 1M tokens

Learn more ↗

GPT-5 mini

A faster, cheaper version of GPT-5 for well-defined tasks

- ✓ Text & vision
- ✓ Reasoning capabilities
- ✓ Support for all built-in tools
- ✓ 400k context length | 128k max output tokens
- ✓ Input: \$0.25 | Output: \$2.00 per 1M tokens

Learn more ↗

GPT-5 nano

The fastest, cheapest version of GPT-5—great for summarization and classification tasks

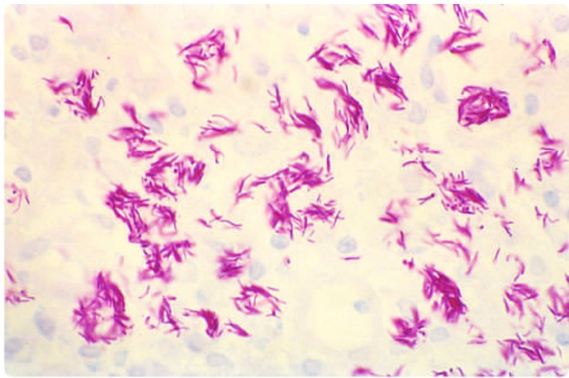
- ✓ Text & vision
- ✓ Reasoning capabilities
- ✓ Support for all built-in tools
- ✓ 400k context length | 128k max output tokens
- ✓ Input: \$0.05 | Output: \$0.40 per 1M tokens

Learn more ↗

<https://github.com/openai/gpt-oss?tab=readme-ov-file>

Instructional Medical Videos: Video Q&A examples

(b) Evaluation Samples



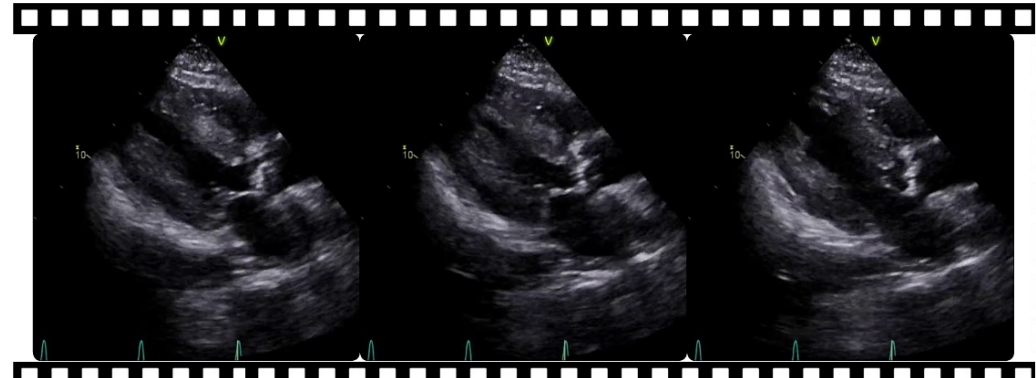
Question: Are there adequate t cells to mount a granulomatous response?

Answer: yes



Question: Are the lungs normal appearing?

Answer: no

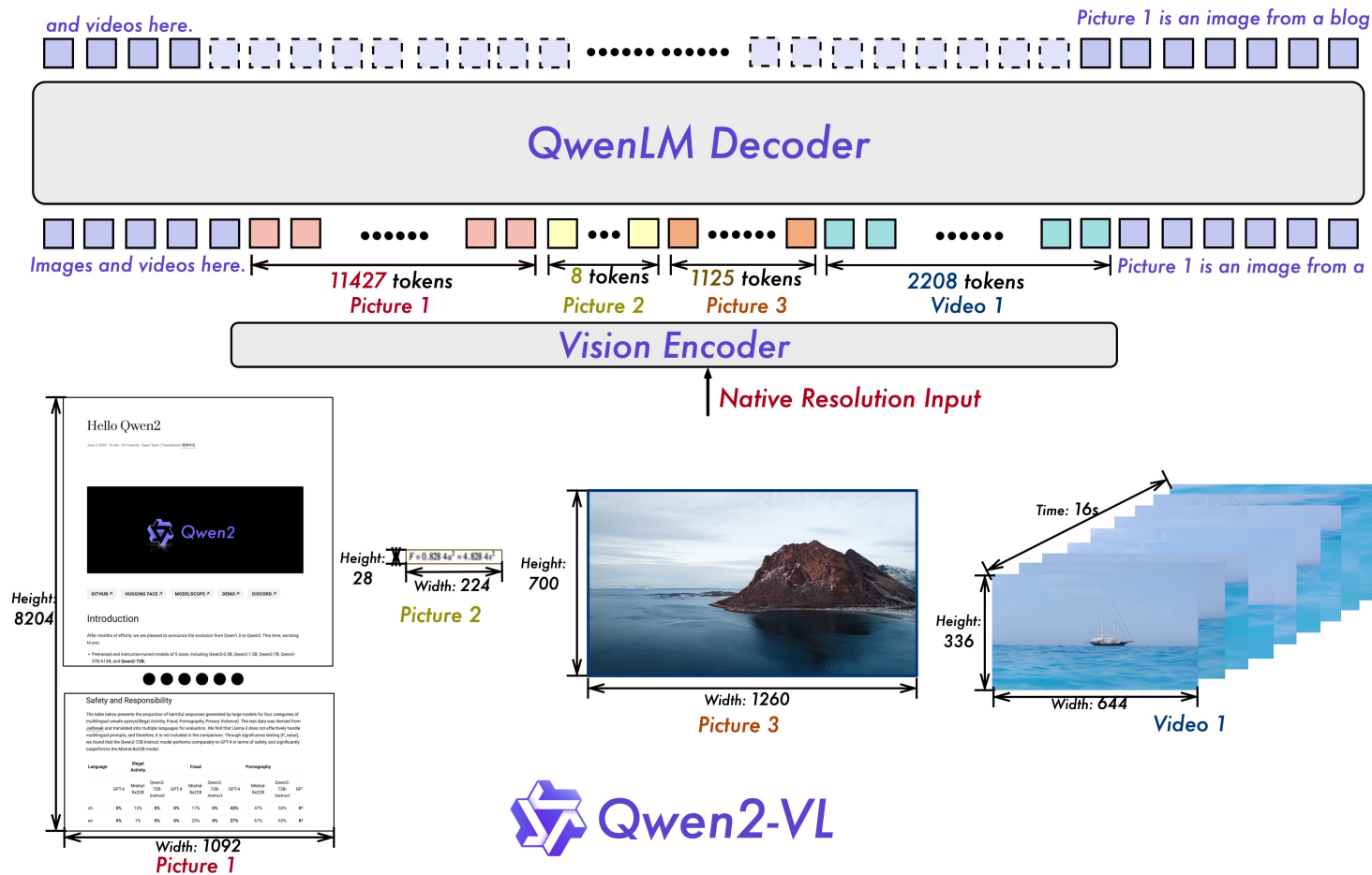


Question: What is the severity of left ventricular hypertrophy?

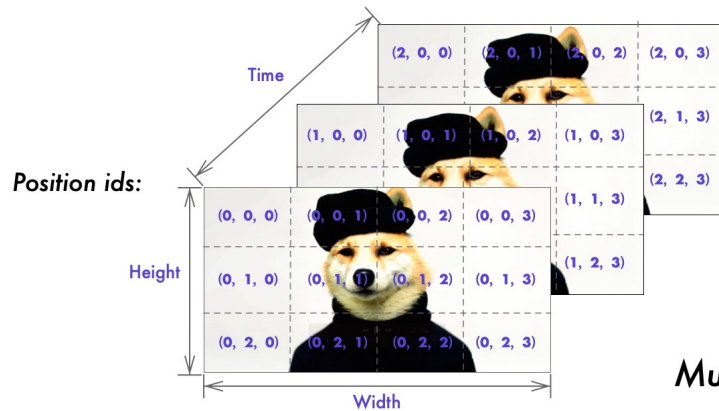
Answer: Mild

How Well Can General Vision-Language Models Learn Medicine By Watching Public Educational Videos?

Qwen2-VL: Tokenizer



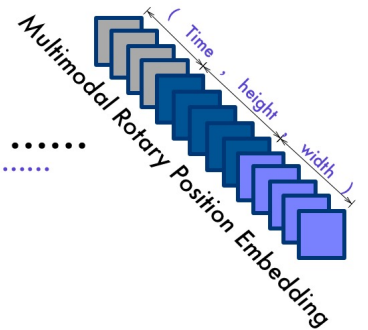
Multimodal Rotary Position Embedding (M-ROPE)



This video features a dog, specifically a Shiba

(4, 4, 4) (5, 5, 5) (6, 6, 6) (7, 7, 7) (8, 8, 8) (9, 9, 9) (10, 10, 10) (11, 11, 11) (12, 12, 12)

Multimodal Rotary Position Embedding (M-ROPE)



Results on VQA Tasks

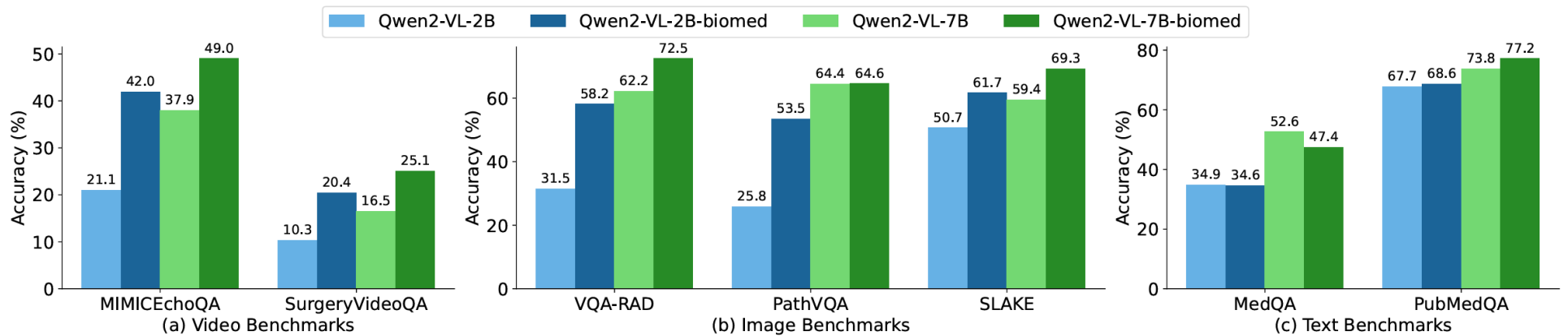


Figure 4: Comparison of fine-tuned models and baseline models across video, image, and text benchmarks.

Concluding remarks

Models

Designs are made up of:

- **Tokenizers, Encoders, Decoders (or Generators), Autoencoders.**

Established Technologies:

- 3D CNNs are easier to train than transformers.
- **Hybrid models** from CNNs and transformers show great promise.
- Clustering for anchor vectors (unsupervised).

Challenges:

- There are **many unexplored video classification models.**
- Foundation models are much **harder (or impossible) to train.**
- **Generative AI models for medical video data augmentation are an unexplored frontier.**

Models: [**https://github.com/pattichis/AIM**](https://github.com/pattichis/AIM)

Datasets (I of II)

Samples of Large Datasets:

- Echocardiography: **EchoNet, MIMIC-IV-ECHO**.
- Endoscopy: (refer to the foundation model for endoscopy).
- Instructional videos: **OpenBiomedVid**.
- Others, see **<https://github.com/pattichis/AIM>**

Lack of ground truth on small or large datasets?

- Data augmentation.
- Adversarial learning.
- Contrastive learning with image transformations.
- Simulation with domain adaptation.

Datasets (II of II)

Lack of ground truth on large datasets (possibly originating from expanding small datasets)?

- Active learning.
- Use a foundation model:
 - Use it to provide ground truth, then verify the output.
 - If the model is open, use knowledge distillation.
- Self-supervised learning:
 1. Train an encoder using an auto-encoder or imputation.
 2. Fine-tune encoder to a specific task.
- GAN: Generate a dataset with the same distribution as the original.
- Generative Models based on diffusion models.